



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий
Кафедра вычислительной техники

РАБОТА ДОПУЩЕНА К ЗАЩИТЕ

Заведующий
Кафедрой _____

 О.В. Платонова

«01» _____ июня 2026 г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
по направлению подготовки бакалавров 09.03.04 Программная инженерия

На тему: Интеллектуальная система подбора треков по ритму движения пользователя

Обучающийся _____


подпись

Свирская Варвара Александровна
Фамилия, имя, отчество

шифр 22И1781
группа ИКБО-22-22

Руководитель работы _____


подпись

к.т.н., доцент

Лозовский В.В.

Москва 2026 г.



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий

Кафедра вычислительной техники

СОГЛАСОВАНО

Заведующий
кафедрой

подпись

Платонова Ольга Владимировна

«27»

февраля

2026 г.

УТВЕРЖДАЮ

И.о. директора
института

подпись

Уланский Александр Борисович

«27»

февраля

2026 г.

ЗАДАНИЕ

на выполнение выпускной квалификационной работы бакалавра

Обучающийся

Свирская Варвара Александровна

Фамилия Имя Отчество

Шифр

22И1781

Направление
подготовки

09.03.04

индекс направления

Программная инженерия

наименование направления

Группа

ИКБО-22-22

1. Тема выпускной квалификационной работы: Интеллектуальная система подбора треков по ритму движения пользователя

2. Цель и задачи выпускной квалификационной работы

Цель работы: разработка интеллектуальной системы, автоматически подбирающей музыкальные треки в соответствии с ритмом движения пользователя.

Задачи работы: изучить методы определения BPM по данным движения и способы описания музыкальных композиций по таким характеристикам, как темп, энергичность и танцевальность, спроектировать архитектуру системы подбора треков на основе машинного обучения, разработать и обучить нейросетевую модель оценки соответствия трека ритму движения пользователя, реализовать программный комплекс для анализа движения, музыкальной базы и рекомендаций, провести тестирование системы на данных MotionSense и каталоге треков, а также оценить точность её работы, практическую ценность и возможности дальнейшего улучшения.

3. Этапы выпускной квалификационной работы

№ этапа	Содержание этапа выпускной квалификационной работы	Результат выполнения этапа ВКР	Срок выполнения
1	Исследовательский раздел		21.03.2026
1.1	Анализ предметной области		
1.2	Обзор существующих аналогов		
1.3	Цель и задачи		
1.4	Техническое задание		
1.5	Вывод по исследовательскому разделу		
2	Аналитический раздел		18.04.2026
2.1	Формальное определение методов решения		
2.2	Обоснование выбора инструментов реализации		
2.3	Анализ процессов системы		
2.4	Анализ архитектуры системы		
2.5	Вывод по аналитическому разделу		
3	Технологический раздел		16.05.2026
3.1	Описание программной реализации		
3.2	Тестирование приложения		
3.3	Расчёт надежности		
3.4	Руководство пользователя		
3.5	Вывод по технологическому разделу		
4	Введение, заключение, список источников, приложения		18.05.2026
5	Презентационный материал, аннотация		27.05.2026
6	Нормоконтроль		28.05.2026

4. Перечень разрабатываемых документов и графических материалов: электронная версия выпускной квалификационной работы бакалавра, презентационный материал с основными результатами выпускной квалификационной работы бакалавра модели процессов подбора музыкальных треков, представленные в нотациях BPMN 2.0 и DFD, схемы данных (логическая и физическая модели), макеты экранных форм пользовательского интерфейса, формы выходных данных, сценарий взаимодействия пользователя с системой, а также графический материал по исследованию предметной области и архитектуре системы

5. Руководитель и консультанты выпускной квалификационной работы

Функциональные обязанности	Должность в Университете	Фамилия, имя, отчество	Подпись
Руководитель ВКР	Доцент	Лозовский Владимир Валерьевич	

Задание выдал

Руководитель ВКР:  подпись

«27» февраля 2026 г.

Задание принял к исполнению

Обучающийся:  подпись

«27» февраля 2026 г.

АННОТАЦИЯ

Выпускная квалификационная работа посвящена разработке интеллектуальной системы подбора музыкальных треков по ритму движения пользователя. Система анализирует данные о движении человека и музыкальные характеристики треков, после чего формирует рекомендации, обеспечивающие совпадение темпа активности и темпа музыкального сопровождения.

В основе системы лежит использование данных открытого датасета MotionSense, методов обработки временных рядов и гибридного подхода, объединяющего алгоритмический расчёт параметров движения и нейросетевую модель, оценивающую степень соответствия между характеристиками движения пользователя и музыкальными признаками треков.

Работа состоит из 110 страниц, включает 28 рисунков и 18 таблиц.

Ключевые слова: интеллектуальная система, машинное обучение, нейронные сети, ВРМ, музыкальные рекомендации, анализ движений.

СОДЕРЖАНИЕ

СПИСОК ИСПОЛЬЗОВАННЫХ СОКРАЩЕНИЙ И ТЕРМИНОВ	6
ВВЕДЕНИЕ	8
ЛИТЕРАТУРНЫЙ ОБЗОР	11
1 ИССЛЕДОВАТЕЛЬСКИЙ РАЗДЕЛ	13
1.1 Анализ предметной области	13
1.2 Обзор существующих аналогов	15
1.3 Цель и задачи	17
1.4 Техническое задание	18
1.5 Вывод по исследовательскому разделу	23
2 АНАЛИТИЧЕСКИЙ РАЗДЕЛ	25
2.1 Формальное определение методов решения	25
2.2 Обоснование выбора инструментов реализации	28
2.3 Анализ процессов системы	30
2.4 Анализ архитектуры системы	35
2.5 Вывод по аналитическому разделу	37
3 ТЕХНОЛОГИЧЕСКИЙ РАЗДЕЛ	39
3.1 Описание программной реализации	39
3.2 Тестирование приложения	43
3.3 Расчёт надёжности	47
3.4 Руководство пользователя	50
3.5 Вывод по технологическому разделу	60
ЗАКЛЮЧЕНИЕ	62
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ	66
ПРИЛОЖЕНИЯ	70

СПИСОК ИСПОЛЬЗОВАННЫХ СОКРАЩЕНИЙ И ТЕРМИНОВ

Датасет – упорядоченный и структурированный набор данных, предназначенный для анализа, обработки и обучения моделей машинного обучения.

MotionSense – открытый набор данных, содержащий временные ряды, записанные акселерометром смартфона.

Прейлист – это структурированный набор музыкальных треков, объединённых по определённому признаку или предназначенных для последовательного воспроизведения.

BPM (Beats Per Minute) – удары в минуту; показатель темпа как в музыке, так и в движениях человека.

Акселерометр – сенсор, измеряющий ускорение объекта по трём взаимно перпендикулярным осям. Он фиксирует линейные изменения скорости движения и позволяет определять характер и интенсивность перемещений, вибраций и колебаний.[1]

PyTorch – открытая библиотека машинного обучения, предназначенная для разработки, обучения и инференса нейронных сетей. Используется в системе для реализации и обучения модели MatchNet.

SciPy – научная библиотека Python, включающая модули обработки сигналов, статистики и вычислительной математики. В работе применяется для фильтрации сенсорных данных и выделения шагов по акселерометру.[2]

NumPy – базовая библиотека Python для работы с многомерными массивами и быстрого численного вычисления. Используется для обработки сигналов движения и векторных операций.[3]

Streamlit – библиотека Python для создания интерактивных веб-приложений, ориентированных на визуализацию данных и работу с моделями машинного обучения. Применяется для реализации пользовательского интерфейса системы.[4]

MatchNet – разработанная в рамках работы нейросетевая модель, выполняющая оценку степени соответствия музыкального трека ритму движения пользователя и используемая в составе гибридного подхода.[5]

Pandas – библиотека Python для работы с табличными данными, используемая для загрузки, фильтрации, очистки и анализа музыкальной базы и данных движения.

Matplotlib – библиотека Python для визуализации данных; применяется для построения графиков и анализа результатов обработки сигналов и вычислений.[6]

TensorFlow – библиотека Python для разработки и обучения моделей машинного обучения; может использоваться для построения и обучения нейросетевых моделей.[7]

nn.Linear – модуль библиотеки PyTorch, реализующий полносвязный (линейный) слой нейронной сети; используется для преобразования входных признаков.

ReLU – функция активации, применяемая в нейронных сетях для введения нелинейности и повышения выразительной способности модели.

Dropout – метод регуляризации нейронных сетей, заключающийся в случайном отключении части нейронов в процессе обучения для предотвращения переобучения.

Sigmoid – функция активации, преобразующая выход нейронной сети в диапазон от 0 до 1; используется для получения вероятностной оценки.

Adam – алгоритм оптимизации, применяемый для обучения нейронных сетей; обеспечивает адаптивную настройку скорости обучения для каждого параметра модели.[8]

CPU (Central Processing Unit) – центральный процессор компьютера; отвечает за выполнение вычислений при обработке данных и работе модели.

GPU (Graphics Processing Unit) – графический процессор, ускоряющий обучение нейросетевых моделей за счёт параллельных вычислений (используется на этапе обучения MatchNet).

ВВЕДЕНИЕ

В современном мире наблюдается значительный рост интереса к персонализированным цифровым сервисам, которые адаптируются под состояние, поведение и активность пользователя. Одной из перспективных областей является подбор музыкального сопровождения, соответствующего темпу движений человека. Музыка выступает не только элементом развлечения, но и инструментом повышения мотивации, улучшения эмоционального состояния и оптимизации физических нагрузок [9].

Тем не менее на практике недостаточно внедрены автоматизированные методы точного подбора треков в соответствии с темпом движений пользователя [10]. Системы, способные сопоставлять темп активности и характеристики музыкальных произведений, пока находятся в зачаточном состоянии, что ограничивает уровень персонализации и эффективность музыкального сопровождения [11].

Актуальность исследования заключается в необходимости создания интеллектуальной системы, способной автоматически анализировать данные движения, сопоставлять их с характеристиками музыкальных треков и формировать персонализированные рекомендации. Использование музыкальных композиций, соответствующих текущей активности пользователя, позволяет оптимизировать нагрузку, поддерживать эмоциональное состояние и улучшать общее качество пользовательского опыта.

Объект исследования — данные о движении человека и характеристики музыкальных треков.

Предмет исследования — процессы анализа, хранения и обработки данных о движении с целью синхронизации музыкального сопровождения с активностью пользователя.

Цель работы — разработка интеллектуальной системы, автоматически подбирающей музыкальные треки в соответствии с ритмом движения пользователя.

Для достижения поставленной цели решаются следующие задачи:

1. Изучить современные методы определения ВРМ на основе данных движения и проанализировать ключевые техники обработки и анализа сенсорных сигналов.
2. Исследовать существующие способы описания музыкальных композиций и методики получения их ритмических и экспрессивных характеристик, включая темп, уровень энергичности и танцевальность.
3. Спроектировать архитектуру системы подбора треков, основанную на принципах машинного обучения.
4. Разработать и обучить нейросетевую модель, а также реализовать базовую эвристическую функцию оценки соответствия музыкального трека ритму движения пользователя.
5. Реализовать программный комплекс, включающий модули анализа движения, музыкальной базы и рекомендаций.
6. Провести тестирование системы на основе данных MotionSense и подготовленного каталога треков из разных жанров музыки.
7. Оценить точность работы системы, определить её практическую ценность и направление дальнейшего улучшения.

Таким образом, работа направлена на создание инструмента, способного автоматически адаптировать музыкальные рекомендации под состояние активности пользователя, облегчить процесс выбора треков и повысить эффективность физической активности.

Для реализации системы в работе используется каскадная модель жизненного цикла программных средств. Разработка выполняется в соответствии с ГОСТ Р ИСО/МЭК 12207–2010 «Процессы жизненного цикла программных средств» [12].

В Приложении А представлено организационно-экономическое обоснование выпускной квалификационной работы.

В Приложении Б представлен графический материал (презентация) выпускной квалификационной работы.

В Приложении В представлен листинг программного кода выпускной квалификационной работы.

В процессе написания выпускной квалификационной работы автор руководствовался следующими нормативными актами:

1. «О защите населения и территории от чрезвычайных ситуаций природного и техногенного характера» от 21.12.1994 № 68-ФЗ (с изменениями, в действующей редакции с 26.11.2024) [13].
2. «Об основах охраны здоровья граждан в Российской Федерации» от 21.11.2011 № 323-ФЗ (с изменениями на 31.07.2025) [14].
3. «О гражданской обороне» от 12.02.1998 № 28-ФЗ (с изменениями на 23.07.2025) [15].
4. Приказ Минздравсоцразвития РФ от 03.05.2024 № 220н «Об утверждении Порядка оказания первой помощи» [16].
5. Трудовой кодекс Российской Федерации от 30.12.2001 № 197-ФЗ (с изменениями на 09.02.2026) [17].
6. Санитарные правила СП 2.2.3670-20 «Санитарно-эпидемиологические требования к условиям труда» [18].
7. СанПиН 1.1.3685-21 «Гигиенические нормативы и требования к обеспечению безопасности и (или) безвредности для человека факторов среды обитания» [19].

ЛИТЕРАТУРНЫЙ ОБЗОР

Исследуемая область объединяет несколько направлений современной науки и техники: анализ данных движения человека, обработку сенсорных сигналов, методы извлечения темпа (BPM), а также технологии представления и анализа музыкальных данных [20]. В работе используются подходы из областей машинного обучения, обработки временных рядов, музыкальной информации и рекомендательных систем.

Одним из ключевых аспектов является анализ сигналов движения. Методы обработки данных, получаемых от акселерометров, подробно рассматриваются в работах по анализу и фильтрации временных рядов. В книге Ханамото, К. «Сенсорные данные и методы их обработки» [21] представлены современные подходы к фильтрации шумов, извлечению ритмических характеристик и выделению периодических компонентов движения. Эти методы лежат в основе алгоритмов определения частоты шагов и расчёта BPM по данным MotionSense.

Немаловажным является и направление музыкальной информатики, занимающееся извлечением ритмических и акустических признаков треков. В книге Мюллер, М. «Fundamentals of Music Processing» [22] детально описаны методы определения темпа музыкальных произведений, анализ ритмической структуры и вычисление таких характеристик, как энергичность и танцевальность. Эти подходы позволяют формировать базу треков с объективными музыкальными параметрами.

Для разработки рекомендательных систем значительный интерес представляют исследования в области интеллектуального подбора контента. Работы Риккарди, Т. и Ким, С. «Music Recommendation Based on Context and User Behavior» [23] рассматривают алгоритмы рекомендаций, основанные на контексте пользователя, включая физическую активность, настроение и окружающую среду. Представленные методы демонстрируют эффективность

адаптивного подбора музыки в зависимости от состояния человека, что непосредственно связано с задачей синхронизации треков с ритмом движений.

Машинное обучение и гибридные методы, объединяющие нейросетевые модели и эвристические алгоритмы, играют ключевую роль в построении моделей, способных прогнозировать степень соответствия музыкального трека активности пользователя. В книге Бишоп, К. «Pattern Recognition and Machine Learning» [24] представлены фундаментальные методы построения обучаемых моделей и работы с многомерными признаками. Эти подходы позволяют формировать информативные признаки, учитывающие разницу BPM, энергичность трека и характеристики движения, которые используются как в нейросетевой модели, так и в базовой эвристической оценке.

Таким образом, анализ литературы показывает, что задачи обработки данных движения, определения BPM и музыкального анализа активно исследуются в смежных областях. Научные источники подтверждают применимость методов машинного обучения, нейросетевых моделей и гибридных подходов для построения интеллектуальных систем рекомендаций, ориентированных на физическую активность пользователя. Результаты этих исследований становятся теоретической основой данной выпускной квалификационной работы.

1 ИССЛЕДОВАТЕЛЬСКИЙ РАЗДЕЛ

В данном разделе проведено исследование предметной области, рассмотрены существующие аналоги и решения, а также сформулированы цели разработки и требования к системе.

1.1 Анализ предметной области

В настоящей выпускной квалификационной работе подбор музыкальных треков на основе ритма движения пользователя рассматривается как задача, объединяющая обработку данных о физической активности и технологии музыкальной информатики. Существует ряд особенностей и сложностей, которые необходимо учитывать при создании интеллектуальной системы, ориентированной на синхронизацию темпа музыки с движением человека:

- формат данных: набор данных содержит многомерные временные ряды, включающие значения акселерометра (x , y , z). Эти данные необходимо обработать, чтобы извлечь устойчивые признаки ритма [25];
- отсутствие данных в режиме реального времени: информация о движении не поступает непрерывным потоком от сенсора, поэтому динамика активности пользователя должна воспроизводиться на основе заранее записанных сессий из набора данных;
- различия между активностями: активности в наборе данных чётко размечены, что позволяет выделять паттерны шагов при ходьбе и беге, но требует корректной интерпретации для последующего использования в системе рекомендаций;
- зашумлённость сенсорных данных: даже в готовом наборе данных присутствуют шумы, характерные для данных акселерометра [26]. Чтобы корректно определить BPM, необходимо применять методы обработки сигналов: фильтрацию, сглаживание и выделение периодических компонентов.

Особенности анализа музыкальных характеристик используемых треков:

- некоторые треки могут восприниматься на разных уровнях темпа: темп может восприниматься как удвоенный или уменьшенный вдвое, что требует нормализации при анализе [27];
- разнообразие музыкальных параметров: помимо BPM система учитывает такие признаки, как энергичность и танцевальность, что повышает точность совпадения музыкального ритма с движением;
- необходимость формирования расширенной музыкальной базы: музыкальные треки разных жанров содержат разнотипные характеристики и требуют унификации по признакам (BPM, энергичность, танцевальность, жанр, исполнитель). Это обеспечивает корректное сопоставление ритма движения пользователя с параметрами музыки.

Особенности разработки системы подбора треков:

- корректное извлечение темпа движения: корректное вычисление BPM из временных рядов набора данных;
- сопоставление ритма движения и свойств треков: необходимо обеспечить согласованность вычисленного темпа активности с музыкальными характеристиками, включая ритмическую структуру и энергичность композиций [28];
- корректное сопоставление ритма: система ориентируется на стабильный BPM, вычисленный на основе выбранного пользователем файла активности;
- учёт пользовательских предпочтений: система предоставляет возможность выбирать желаемый жанр, фильтровать исполнителей и учитывать предпочтения по энергичности. Это требует корректной логики фильтрации данных до этапа ранжирования;
- построение интегрированной модели: требуется объединить данные о движении с параметрами музыкальных произведений в единой модели, обеспечивающей надёжную оценку соответствия и формирование рекомендаций.

1.2 Обзор существующих аналогов

Современные цифровые платформы активно используют методы персонализации, включая рекомендации музыки, однако систем, которые автоматически подстраивают музыкальное сопровождение под физическую активность пользователя, сравнительно мало [29]. Существующие решения либо ограничены функциональностью, либо недоступны для адаптации под научные или исследовательские задачи. Ниже рассмотрены наиболее известные аналоги и их ключевые достоинства и недостатки:

1. Spotify Running

Spotify Run (уже закрытая функция) определяла темп бега пользователя и подбирала музыку соответствующего BPM.

Преимущества:

- автоматическое определение темпа бега;
- большой каталог музыки;
- стабильная работа на мобильных устройствах.

Недостатки:

- закрытая технология, недоступная для исследовательских задач;
- подбор ограничен лишь BPM — отсутствует анализ энергичности, танцевальности, ритмической структуры;
- система не обучаемая, отсутствует интеллектуальный анализ (включая методы машинного обучения).

2. RockMyRun

Приложение формирует динамические миксы, которые могут подстраиваться под темп движения.

Преимущества:

- возможность частичного синхронизирования музыки с ритмом бега;
- наличие готовых миксов под разные активности.

Недостатки:

- отсутствие точного определения реального BPM движения;

- нет нейросетевой оценки соответствия трека активности;
- нет фильтрации по жанру, артистам или дате выпуска;
- пользователю недоступна возможность гибкой кастомизации рекомендаций.

3. PaceDJ

Позволяет пользователю вручную подобрать BPM трека под тренировку.

Преимущества:

- простая настройка;
- удобный подбор плейлистов по темпу.

Недостатки:

- отсутствие автоматического вычисления темпа движения пользователя;
- полная зависимость от ручного ввода BPM;
- отсутствие интеллектуальной оценки соответствия трека реальной физической активности;
- отсутствуют дополнительные фильтры (жанр, артист).

Ниже представлены сравнительные характеристики существующих аналогов и разрабатываемой системы (Таблица 1.1).

Таблица 1.1 — Сравнительная характеристика аналогов и предлагаемой системы

Система	Определение BPM движения	Подбор по BPM	Учёт энергии	Учёт танцевальности	Жанровые предпочтения	Фильтрация по артистам	Доступность технологий
Spotify Running	Да	Да	Нет	Нет	Нет	Нет	Нет
RockMy Run	Нет	Да	Нет	Нет	Нет	Нет	Нет
PaceDJ	Нет	Да	Нет	Нет	Нет	Нет	Да
Разрабатываемая система	Да	Да	Да	Да	Да	Да	Да

Несмотря на наличие сервисов, предоставляющих музыку для тренировок, ни одно из существующих решений не сочетает одновременно:

- точное определение BPM движения по данным сенсоров;
- совместный анализ BPM, энергичности и танцевальности трека;
- интеллектуальное сопоставление движения и музыки;
- жанровые предпочтения пользователя;
- предпочтение по артистам и возможность исключения артистов.

Разрабатываемая система устраняет указанные ограничения, сочетая анализ движения, музыкальных признаков и пользовательских предпочтений в единой модели рекомендаций. Благодаря открытой архитектуре и расширяемому каталогу система может использоваться как в исследовательских работах, так и в практических пользовательских сценариях.

В результате система обеспечивает более глубокий и точный подбор музыки, чем любые существующие решения.

1.3 Цель и задачи

Целью разработки является создание интеллектуальной программной системы, автоматически подбирающей музыкальные треки в соответствии с ритмом движения пользователя, определённым на основе данных сенсорных сигналов.

Для достижения поставленной цели решаются следующие задачи:

1. Изучить современные методы определения BPM по данным движения, а также исследовать алгоритмы обработки и анализа акселерометрических сигналов для выделения устойчивых ритмических компонентов.

2. Исследовать существующие способы описания музыкальных композиций и методики получения ритмических и выразительных характеристик треков, включая темп, энергичность и танцевальность.

3. Спроектировать архитектуру интеллектуальной системы подбора треков, основанную на модульном подходе и принципах машинного обучения [30].

4. Разработать и обучить нейросетевую модель, а также реализовать базовую эвристическую функцию оценки соответствия музыкального трека ритму движения пользователя.

5. Реализовать программный комплекс, включающий модули анализа движения, обработки музыкальных данных и формирования рекомендаций, а также интегрировать их в единую систему.

6. Провести тестирование системы на данных MotionSense и подготовленной музыкальной базе, оценив точность рекомендаций и корректность работы всех модулей.

7. Выполнить анализ эффективности работы системы, определить степень практической применимости и сформулировать возможные направления дальнейшего развития.

Решение этих задач обеспечивает создание интеллектуальной системы, способной автоматически анализировать данные движения, сопоставлять их с музыкальными характеристиками и формировать персонализированные рекомендации. Это повышает удобство подбора музыкального сопровождения и способствует улучшению качества физической активности пользователя.

1.4 Техническое задание

В данном подразделе представлено техническое задание на разработку интеллектуальной системы подбора музыкальных треков по ритму движения пользователя. Техническое задание разработано на основе требований ГОСТ 19.201–78 [31].

1.4.1 Функциональные требования

1. Модуль анализа движения пользователя:

- загрузка данных сенсоров (акселерометра) из датасета MotionSense;

- предобработка сигналов;
 - вычисление BPM движения пользователя.
2. Модуль анализа музыкальных треков:
- загрузка музыкального каталога;
 - извлечение характеристик треков;
 - приведение музыкальных характеристик к единому формату;
 - предварительная фильтрация треков по пользовательским предпочтениям (жанр, артисты, энергичность);
 - формирование единой базы музыкальных признаков.
3. Модуль гибридной оценки соответствия:
- обработка признаков движения и музыкальных характеристик трека;
 - вычисление интегральной оценки соответствия трека ритму движения пользователя с помощью нейросетевой модели и базовой эвристической функции;
 - обучение и переобучение нейросети на размеченных данных для улучшения качества рекомендаций;
 - передача рассчитанных оценок соответствия в модуль рекомендаций.
4. Модуль рекомендаций:
- ранжирование треков по итоговой оценке и формирование списка рекомендаций по выбранным критериям (жанр, предпочтительные артисты, артисты, исключённые из рекомендаций, желаемый стиль треков по энергичности и танцевальности);
 - приём интегральных оценок соответствия треков, рассчитанных нейросетевой моделью;
 - ранжирование треков по степени соответствия ритму движения и заданным предпочтениям пользователя;
 - формирование финального списка треков для пользователя.
5. Вывод результатов работы системы:

- отображение текущего рассчитанного BPM движения;
- вывод списка рекомендованных треков;
- отображение ключевых характеристик музыки;
- предоставление удобной и понятной визуализации (Web-интерфейс);
- отображение возможных ошибок и предупреждений;
- предоставление возможности сохранения полученных рекомендаций в CSV-файл.

1.4.2 Требования к надежности

Для обеспечения корректной и стабильной работы программной системы формируются следующие требования:

- корректно обрабатывать шумные и неполные данные сенсоров;
- обеспечивать устойчивость при некорректных значениях в датасете;
- гарантировать стабильность работы при многократных анализах и перерасчётах;
- иметь механизм проверки данных на валидность перед обработкой.

1.4.3 Требования к совместимости

Для корректной работы разрабатываемого программного обеспечения необходимы следующие программные компоненты:

- язык разработки: Python 3.8+;
- библиотеки: NumPy, Pandas, SciPy, PyTorch;
- поддержка операционных систем: Windows, macOS, Linux.

Для функционирования системы не требуется специализированного оборудования. Минимальные аппаратные требования включают:

- нормальная работа на персональных компьютерах средней мощности;

- отсутствие требований к графическому процессору, кроме этапа обучения модели.

1.4.4 Требования к документации

Для разработки и последующей эксплуатации системы необходимо подготовить комплект программной документации, включающий следующие материалы:

- техническое задание;
- описание архитектуры программы;
- описание алгоритмов обработки данных движения;
- описание модели рекомендаций;
- пояснительная записка.

1.4.5 Требования к эргономике

Интерфейс системы должен соответствовать ряду требований, обеспечивающих удобство и корректность взаимодействия пользователя с программным продуктом:

- простым, интуитивным и минималистичным;
- удобным для пользователя без специальных технических знаний;
- обеспечивать наглядное отображение рассчитанного ВРМ движения, выбранных параметров фильтрации и сформированного списка рекомендованных треков;
- поддерживать удобный выбор пользовательских критериев (жанр, предпочтительные артисты, исключённые артисты, желаемый уровень энергичности);
- корректно отображать предупреждения и ошибки при некорректных или неполных данных, включая подсказки по устранению проблемы;

- адаптированным под формат веб-интерфейса и корректно отображаться в браузере при стандартных разрешениях экрана ПК.

1.4.6 Требования к программной документации

Предварительный состав программной документации должен включать в себя:

- техническое задание;
- текст программы;
- документации;
- описание программы;
- пояснительная записка;
- описание применения;
- руководство пользователя.

1.4.7 Этапы разработки

Разработка интеллектуальной системы подбора музыкальных треков по ритму движения пользователя включает следующие этапы:

1. Анализ предметной области и существующих решений.
2. Формирование технического задания.
3. Проектирование архитектуры системы.
4. Разработка алгоритмов обработки данных движения.
5. Формирование музыкальной базы и извлечение характеристик треков.
6. Разработка и обучение нейросетевой модели.
7. Интеграция модулей в единый программный комплекс.
8. Тестирование системы и анализ результатов.

1.5 Вывод по исследовательскому разделу

В ходе исследовательского раздела была изучена предметная область интеллектуальных систем подбора музыкальных треков, синхронизированных с ритмом движения пользователя. Проведённый анализ показал, что задача согласования темпа музыки с физической активностью требует комплексного подхода, объединяющего методы обработки сенсорных данных и анализа музыкальных характеристик.

На первом этапе исследования были рассмотрены принципы получения и обработки данных о движении пользователя, включая анализ сигналов акселерометра, представленных в наборе данных MotionSense. Установлено, что данные движения являются многомерными временными рядами, требующими предварительной фильтрации, выделения периодических компонентов и последующего анализа. Было определено, что корректное вычисление BPM возможно только при использовании методов обработки сигналов, позволяющих выделять устойчивые интервалы между шагами и снижать влияние шумов.

Далее был проведён анализ музыкальных данных, включающих характеристики треков, такие как темп, энергичность и танцевальность. Выявлена необходимость приведения данных к единому формату и использования данных признаков для сопоставления с параметрами движения пользователя. Также установлено, что дополнительные характеристики, такие как жанр и исполнитель, целесообразно использовать на этапе фильтрации, а не в составе модели оценки.

Анализ существующих аналогов показал, что современные системы либо ограничиваются простым сопоставлением по BPM, либо используют заранее сформированные плейлисты, либо не предоставляют доступа к внутренним алгоритмам. Ни одно из рассмотренных решений не объединяет точное определение темпа движения по сенсорным данным, анализ нескольких музыкальных характеристик и интеллектуальную модель оценки соответствия.

В результате исследования была обоснована необходимость разработки собственной системы, использующей гибридный подход, объединяющий нейросетевую модель и эвристическую функцию оценки соответствия. Такой подход позволяет повысить точность и устойчивость рекомендаций по сравнению с существующими решениями.

На основании проведённого анализа были сформулированы цель и задачи разработки, определены основные этапы построения системы и её архитектурные принципы. Итогом исследовательского раздела стало формирование теоретической и методологической основы, необходимой для реализации интеллектуальной системы подбора музыкальных треков по ритму движения пользователя.

Таким образом, выполненное исследование подтвердило актуальность выбранной темы и позволило определить подходы, обеспечивающие корректное определение параметров движения, анализ музыкальных характеристик и формирование персонализированных рекомендаций на основе гибридной модели оценки.

2 АНАЛИТИЧЕСКИЙ РАЗДЕЛ

В данном разделе рассмотрены методы решения задачи подбора музыкальных треков по ритму движения пользователя, включая математические подходы к обработке сенсорных данных и анализу музыкальных характеристик. Были определены ключевые параметры, используемые для сопоставления движения и треков, и обоснован выбор инструментов реализации. Кроме того, разработана архитектура системы, представленная в виде диаграммы компонентов, отражающей организацию модулей и их взаимодействие.

2.1 Формальное определение методов решения

В основе разработанной системы лежит комплексная обработка временных рядов акселерометрических данных, анализ музыкальных характеристик и применение гибридной модели, объединяющей нейросетевой подход и эвристическую оценку, для вычисления интегральной степени соответствия музыкального трека текущему ритму движения пользователя.

Данные движения пользователя представляют собой многомерные временные ряды $a_x(t)$, $a_y(t)$, $a_z(t)$ получаемые с акселерометра смартфона. Для выделения устойчивой ритмической компоненты используется модуль вектора ускорения $a(t)$, который позволяет преобразовать трёхкомпонентный сигнал к одной аналитически удобной форме. Он вычисляется по формуле 2.1.

$$a(t) = \sqrt{a_x^2(t) + a_y^2(t) + a_z^2(t)}, \quad (2.1)$$

где $a_x(t)$ — проекция ускорения на ось x , полученная с акселерометра;

$a_y(t)$ — проекция ускорения на ось y ;

$a_z(t)$ — проекция ускорения на ось z .

Данный сигнал содержит периодическую структуру, соответствующую шагам пользователя. После применения полосового фильтра Баттерворта [32] в диапазоне 0.5–3.5 Гц выделяется шаговый ритм, а детекция локальных максимумов позволяет определить период движения. Темп активности (BPM) вычисляется на основе медианного интервала между шагами по формуле 2.2.

$$BPM = \frac{60}{\overline{\Delta t}}, \quad (2.2)$$

где $\overline{\Delta t}$ — медианный период между шагами.

Каждый музыкальный трек характеризуется набором признаков, включающим темп (BPM), энергичность и танцевальность. При этом показатели энергичности и танцевальности нормированы в диапазоне [0,1].

Для учёта предпочтений пользователя по интенсивности музыкальных треков вводится показатель согласованности энергичности, рассчитываемый по формуле 2.3.

$$energy_alignment = 1 - |energy - pref|, \quad (2.3)$$

где $energy$ — энергичность трека;

$pref$ — числовое представление предпочтения пользователя.

Для каждой пары «движение пользователя — музыкальный трек» формируется объединённый входной вектор (z) который рассчитывается по формуле 2.4.

$$z = (\widehat{bpm}_{user}, \widehat{bpm}_{track}, |\widehat{bpm}_{user} - \widehat{bpm}_{track}|, energy_alignment, dance), \quad (2.4)$$

где $\widehat{bpm}_{user}$ — нормализованный BPM пользователя;

$\widehat{bpm}_{track}$ — нормализованный BPM музыкального трека;

$|\widehat{bpm}_{user} - \widehat{bpm}_{track}|$ — абсолютная разница темпов;

energy_alignment — нормализованная степень соответствия энергичности трека предпочтениям пользователя;

dance — нормализованная танцевальность трека.

Нормализация признаков, связанных с BPM, выполняется с использованием статистических параметров выборки (среднего значения и стандартного отклонения), что обеспечивает стабильность обучения модели.

Для определения степени соответствия используется нейросетевая модель MatchNet, представляющая собой полносвязную сеть со следующей структурой представленной в формуле 2.5.

$$\begin{aligned}h_1 &= ReLU(W_1 z + b_1), \\h_2 &= Dropout(ReLU(W_2 h_1 + b_2)), \\ \hat{y} &= \sigma(W_3 h_2 + b_3),\end{aligned}\tag{2.5}$$

где ReLU — функция активации;

Dropout — метод регуляризации;

W_1, W_2, W_3 — матрицы весов полносвязных слоёв нейронной сети;

b_1, b_2, b_3 — векторы смещений, соответствующие каждому слою;

h_1, h_2 — выходные активации промежуточных слоёв модели;

σ — сигмоидальная функция, нормализующая выход в диапазон $[0, 1]$.

Выходное значение ($\hat{y}$) определяется по формуле 2.6.

$$\hat{y} \in [0,1],\tag{2.6}$$

Интерпретируется как интегральная оценка соответствия трека движению пользователя:

$\hat{y} = 1$ — трек идеально подходит по ритму и стилю,

$\hat{y} = 0$ — трек не согласован с движением.

Помимо нейросетевой модели используется базовая эвристическая функция оценки, учитывающая разницу темпа, энергичность и танцевальность

трека. Для повышения устойчивости системы применяется гибридный подход, при котором итоговая оценка формируется как взвешенная комбинация нейросетевого предсказания и базовой оценки. Оценка рассчитывается по формуле 2.7.

$$S = \alpha * \hat{y} + \beta * S_{base}. \quad (2.7)$$

где $\hat{y}$ — предсказание нейросетевой модели;

S_{base} — базовая эвристическая оценка;

α, β — весовые коэффициенты.

Дополнительно в системе реализована адаптация параметров оценки под тип физической активности пользователя (ходьба, бег, подъём и спуск по лестнице). Для каждой активности задаются индивидуальные ограничения на допустимую разницу BPM и минимальное значение итоговой оценки, что повышает точность рекомендаций.

Таким образом, предложенный подход объединяет методы цифровой обработки сигналов, нормализацию признаков и гибридную модель оценки соответствия, что позволяет учитывать как характеристики движения, так и музыкальные параметры и обеспечивает более точный подбор треков по сравнению с традиционными методами, основанными только на темпе.

2.2 Обоснование выбора инструментов реализации

Для реализации системы подбора музыкальных треков по ритму движения пользователя необходимо использовать инструменты, обеспечивающие обработку временных рядов, анализ данных и построение модели машинного обучения.

Для обработки сигналов движения рассматривались библиотеки SciPy и PyTorch (signal processing через torch). Выбор был сделан в пользу библиотеки SciPy, так как она содержит готовые реализации цифровых фильтров, в

частности фильтра Баттерворта, а также функции обработки временных рядов. Использование модуля `scipy.signal` позволяет корректно выделить ритмическую составляющую движения пользователя. Для определения шагов применяется функция `find_peaks`, обеспечивающая детекцию локальных максимумов сигнала.

Поскольку библиотека SciPy является частью экосистемы языка Python, выбор данного инструмента определил использование Python в качестве основного языка программирования.

Для обработки табличных данных рассматривались инструменты Pandas и NumPy. Библиотека NumPy эффективна для выполнения численных операций, однако не предоставляет удобных средств для работы с таблицами и метаданными. В связи с этим для загрузки, объединения и очистки данных была выбрана библиотека Pandas, обеспечивающая удобную работу с CSV-файлами и поддержку операций фильтрации, удаления пропущенных значений и преобразования признаков.

Библиотека NumPy используется совместно с Pandas для выполнения векторизованных вычислений и ускорения обработки данных.

Для реализации модели машинного обучения рассматривались библиотеки TensorFlow и PyTorch. Выбор был сделан в пользу PyTorch, поскольку он предоставляет более гибкий механизм построения архитектуры нейронной сети и удобную работу с динамическими вычислительными графами, что важно при реализации собственной модели.

В рамках выбранного подхода используется полносвязная нейронная сеть, реализованная с помощью слоёв `nn.Linear`. Такой выбор обусловлен тем, что входные данные представлены в виде фиксированного вектора признаков и не обладают пространственной структурой, поэтому применение свёрточных или рекуррентных нейронных сетей является избыточным.

Для введения нелинейности используется функция активации ReLU, позволяющая учитывать сложные зависимости между признаками. Для предотвращения переобучения применяется слой Dropout, который снижает зависимость модели от отдельных нейронов.

Выходной слой реализован с использованием функции Sigmoid, так как задача сводится к вычислению оценки соответствия в диапазоне от 0 до 1.

В качестве оптимизатора используется алгоритм Adam, обеспечивающий устойчивую сходимость и адаптивную настройку параметров обучения.

Для подготовки данных применяется стандартизация признаков, связанных с ВРМ, что позволяет привести их к единому масштабу и повысить стабильность обучения модели.

Дополнительно в системе используется библиотека Streamlit для реализации пользовательского интерфейса и визуализации результатов работы системы.

Таким образом, выбор инструментов обусловлен их функциональными возможностями и соответствием задачам обработки сигналов, анализа данных и построения гибридной модели рекомендаций, что позволяет обеспечить точность и устойчивость работы системы.

2.3 Анализ процессов системы

В рамках анализа процессов системы построена диаграмма потоков данных (DFD), отражающая структуру обработки информации и последовательность преобразований внутри системы подбора музыкальных треков по ритму движения пользователя.

На уровне контекстной модели система рассматривается как единый процесс «Система подбора музыкальных треков по ритму движения». С системой взаимодействуют три внешние сущности:

- пользователь, задающий параметры выбора и получающий результат;
- датасет MotionSense, предоставляющий сырые сенсорные данные движения;
- музыкальный каталог, содержащий характеристики треков (темп, энергичность, танцевальность).

Входные потоки данных включают:

- сырые сенсорные данные акселерометра;
- музыкальные характеристики треков;
- параметры выбора пользователя (тип активности, жанр, любимые и исключённые исполнители, предпочтения по энергичности).

Выходной поток данных представляет собой:

- список рекомендованных треков, отсортированных по степени соответствия + рассчитанное значение ВРМ движения пользователя.

Результат построения контекстной диаграммы представлен на рисунке 2.1.

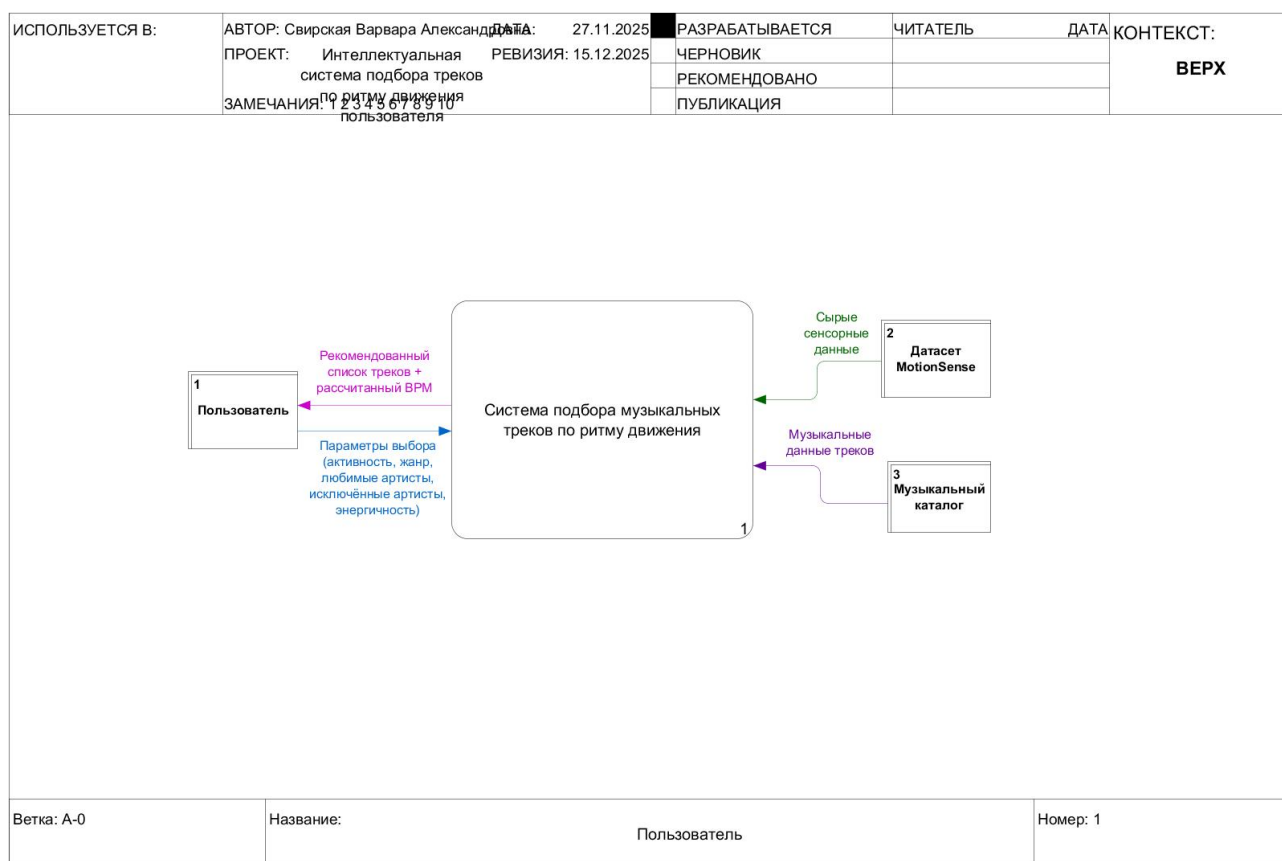


Рисунок 2.1 – Контекстная диаграмма

На следующем уровне диаграмма раскрывает внутреннюю структуру системы и показывает четыре основных функциональных процесса:

1. Анализ движения пользователя.

Назначение:

- процесс предназначен для обработки сырых сенсорных данных и определения параметров двигательной активности пользователя.

Входные данные:

- сырые данные акселерометра из датасета MotionSense.

Выходные данные:

- BPM пользователя + очищенный сигнал;
- запись в хранилище «Промежуточные данные движения».

Описание обработки:

- выполняется преобразование трёхкомпонентного сигнала ускорения в одномерный сигнал, его фильтрация с использованием полосового фильтра Баттерворта и выделение ритмической составляющей. Далее производится детекция шагов и вычисление BPM на основе медианного интервала между шагами, что повышает устойчивость к шумам.

2. Анализ музыкальных треков.

Назначение:

- процесс обеспечивает подготовку и нормализацию признаков музыкальных треков.

Входные данные:

- данные музыкального каталога (BPM, энергичность, танцевальность).

Выходные данные:

- нормализованные признаки треков;
- запись в хранилище «База признаков треков».

Описание обработки:

- выполняется загрузка музыкальных характеристик, их очистка и приведение к единому формату. Признаки, связанные с BPM, нормализуются, а показатели энергичности и танцевальности используются в исходном виде. Формируется набор признаков, используемый на этапе оценки соответствия.

3. Оценка соответствия треков (гибридная модель).

Назначение:

- процесс предназначен для вычисления степени соответствия музыкальных треков параметрам движения пользователя.

Входные данные:

- BPM пользователя из хранилища «Промежуточные данные движения»;
- нормализованные признаки треков из хранилища «База признаков треков»;
- параметры выбора пользователя.

Выходные данные:

- результаты сопоставления треков;
- запись в хранилище «Оценки соответствия треков».

Описание обработки:

- для каждой пары «пользователь — трек» формируется вектор признаков, включающий нормализованный BPM пользователя, нормализованный BPM трека, разницу между BPM пользователя и трека, показатель согласованности энергичности и танцевальности трека;
- полученные векторы подаются в нейросетевую модель MatchNet, которая вычисляет оценку соответствия. Дополнительно рассчитывается базовая эвристическая оценка, основанная на разнице темпа и характеристиках трека;
- итоговая оценка формируется как комбинация нейросетевого предсказания и базовой оценки (гибридный подход), что позволяет повысить устойчивость системы и точность результатов.

4. Оценка уверенности рекомендаций.

Назначение:

- процесс предназначен для определения степени надёжности сформированных рекомендаций на основе анализа полученных оценок соответствия треков и параметров движения пользователя.

Входные данные:

- результаты сопоставления треков из хранилища «Оценки соответствия треков».

Выходные данные:

- уровень уверенности рекомендаций.

Описание обработки:

- на данном этапе выполняется анализ полученных оценок соответствия музыкальных треков. Рассматриваются такие параметры, как максимальное значение оценки, распределение оценок среди выбранных треков, а также разница BPM между пользователем и рекомендованными композициями;

- на основе этих показателей вычисляется уровень уверенности рекомендаций, отражающий степень их соответствия ритму движения пользователя. Это позволяет дополнительно оценить качество сформированных рекомендаций и повысить информативность результата для пользователя.

5. Фильтрация, ранжирование и формирование рекомендаций.

Назначение:

- процесс обеспечивает формирование итогового набора рекомендаций музыкальных треков.

Входные данные:

- треки с оценками соответствия из хранилища «Оценки соответствия треков»;
- уровень уверенности рекомендаций;
- параметры выбора пользователя.

Выходные данные:

- список рекомендованных треков, отсортированных по степени соответствия, рассчитанный BPM.

Описание обработки:

- на данном этапе выполняется предварительная фильтрация треков по пользовательским предпочтениям (жанр, любимые и исключённые исполнители). Далее треки ранжируются по итоговой оценке соответствия, формируется список рекомендаций и передаётся пользователю.

Результат построения декомпозиции процессов представлен на рисунке 2.2.

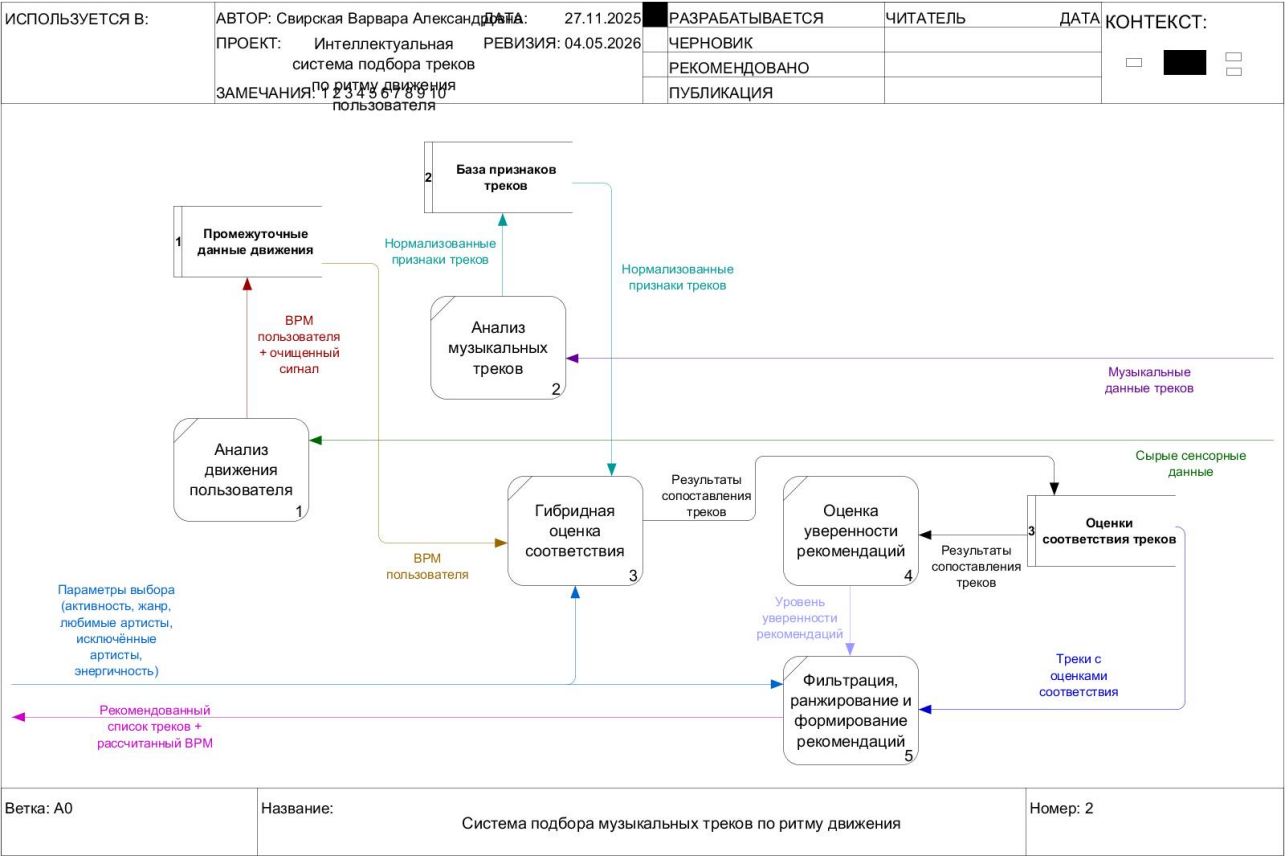


Рисунок 2.2 – Декомпозиция процессов системы

Проведённый анализ показывает, что система обладает чёткой структурой обработки данных с последовательной передачей результатов между функциональными модулями. Каждый процесс выполняет строго определённую функцию и использует соответствующие входные и выходные данные. Применение гибридного подхода, объединяющего нейросетевую модель и эвристическую оценку, позволяет повысить точность и устойчивость системы. Диаграмма потоков данных подтверждает, что итоговые рекомендации формируются на основе комплексной обработки сенсорной информации и музыкальных характеристик, объединённых в единую логическую модель работы системы.

2.4 Анализ архитектуры системы

Архитектурная модель системы отражает ключевые программные компоненты, их функциональные роли и взаимодействие между ними. Построенная UML-диаграмма компонентов позволяет наглядно представить

структуру системы, границы модулей и последовательность обработки данных, начиная с получения исходной информации и заканчивая формированием рекомендаций. Результат построения UML-диаграммы представлен на рисунке 2.3.

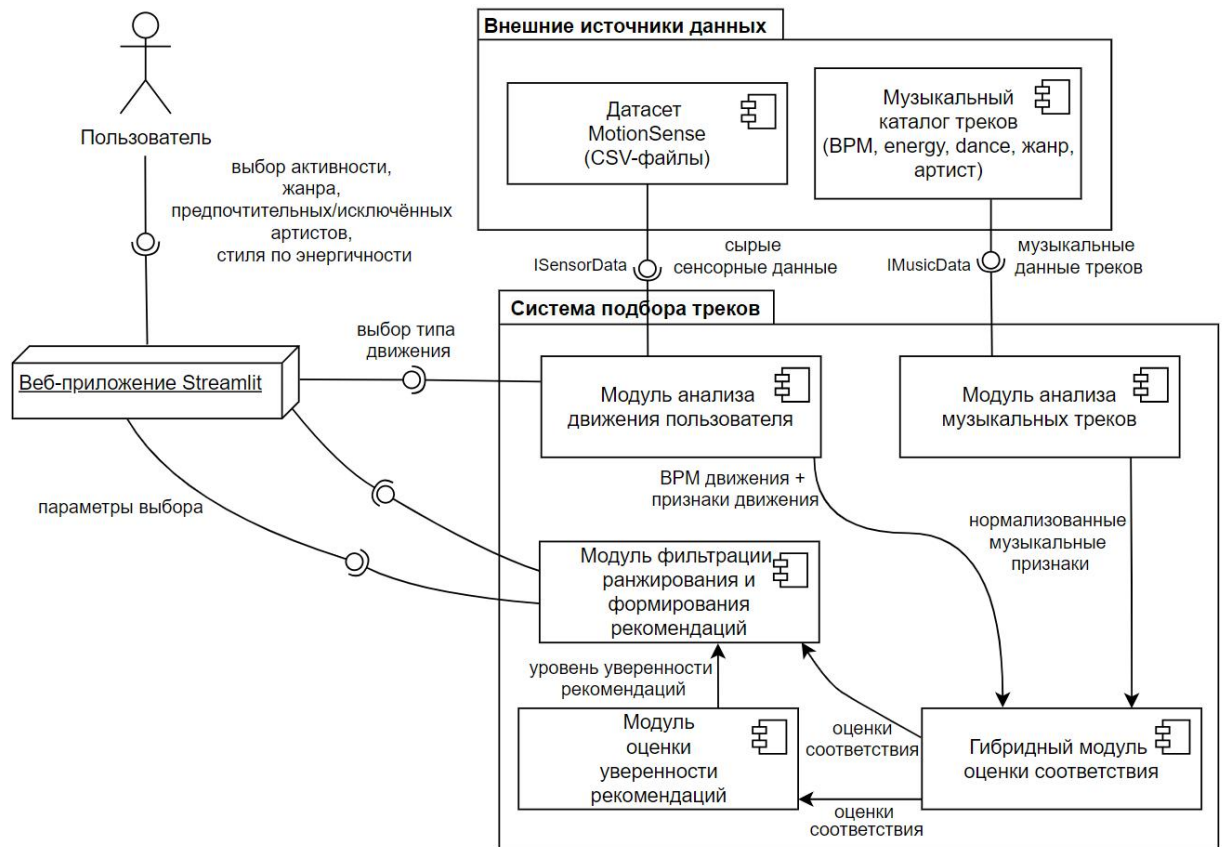


Рисунок 2.3 – UML-диаграмма компонентов системы

Система подбора музыкальных треков по ритму движения пользователя функционирует во взаимодействии с внешними источниками данных, к которым относятся датасет MotionSense, содержащий сырые сенсорные данные в формате CSV, и музыкальный каталог треков, включающий характеристики композиций (BPM, energy, dance, жанр, артист). Пользователь через веб-приложение, реализованное на платформе Streamlit, задаёт параметры выбора, включая тип активности, жанр, предпочтительных и исключённых исполнителей, а также уровень энергичности, а также выбирает тип движения.

Сенсорные данные поступают в модуль анализа движения пользователя, где выполняется их обработка и вычисление BPM, а также формирование признаков движения. Параллельно музыкальные данные из каталога

передаются в модуль анализа музыкальных треков, в котором осуществляется обработка и нормализация признаков композиций.

Полученные признаки движения пользователя и нормализованные признаки музыкальных треков поступают в гибридный модуль оценки соответствия. В данном модуле формируются векторы признаков, объединяющие характеристики движения и треков, после чего вычисляются оценки соответствия с использованием гибридного подхода, включающего нейросетевую модель и базовую эвристическую оценку.

Результаты оценки соответствия передаются в модуль оценки уверенности рекомендаций, где определяется уровень надёжности полученных результатов. Далее оценки соответствия и уровень уверенности поступают в модуль фильтрации, ранжирования и формирования рекомендаций, который также получает параметры выбора от пользователя. В этом модуле осуществляется отбор треков по заданным условиям, их ранжирование и формирование итогового списка рекомендаций.

В результате работы системы формируется список рекомендованных треков и рассчитанный BPM пользователя, которые передаются обратно в веб-приложение и отображаются пользователю.

2.5 Вывод по аналитическому разделу

В ходе аналитического раздела была сформирована целостная концепция разрабатываемой системы и определены подходы, лежащие в её основе. Выполненный анализ позволил уточнить математические методы решения задачи, структуру обработки данных и логику взаимодействия ключевых модулей. Формализованы процессы работы системы и представлена архитектурная модель, демонстрирующая, какие компоненты будут реализованы программно и каким образом они связаны между собой.

Полученные результаты обеспечили полное понимание того, какие функции должна выполнять система, какие алгоритмы используются для

анализа движения и музыкальных характеристик, какие данные участвуют в обработке и как происходит формирование итоговых рекомендаций. Таким образом, аналитический раздел определил технические и методологические основания для последующей реализации программной части и разработки функционирующего прототипа системы.

3 ТЕХНОЛОГИЧЕСКИЙ РАЗДЕЛ

В данном разделе представлено описание программной реализации системы подбора музыкальных треков по ритму движения пользователя. Рассмотрены структура разработанного программного обеспечения, функции основных модулей, используемые алгоритмы и инструменты. Описаны результаты тестирования, обеспечивающие подтверждение корректности и надёжности работы системы, а также представлены сценарии взаимодействия пользователя с приложением.

3.1 Описание программной реализации

Программная реализация системы подбора музыкальных треков по ритму движения пользователя построена в соответствии с архитектурой, описанной в разделе 2.4, и включает набор модулей, каждый из которых отвечает за выполнение определённых бизнес-процессов. Логическая структура программы отражает основные этапы обработки данных: анализ движения, извлечение музыкальных характеристик, вычисление оценки соответствия и формирование рекомендаций. Все функции и файловые сущности разработаны на языке Python с использованием библиотеки PyTorch для построения и работы нейросетевой модели, а также фреймворка Streamlit для реализации веб-интерфейса.

Ниже приведено описание ключевых программных компонентов, структурированных по файлам и функциональным модулям.

1. Файл: `motion_processing.py` — модуль анализа движения пользователя.

Основные функции:

- `load_motion_data(path)` — загружает сырые временные ряды акселерометра из датасета MotionSense. Осуществляет первичную загрузку и подготовку входных данных для дальнейшей обработки;

- `preprocess_signal(data)` — выполняет сглаживание, фильтрацию и удаление шумов. Соответствует бизнес-процессу предобработки сенсорных данных;

- `compute_bpm(data)` — реализует вычисление BPM на основе фильтрации сигнала, детекции шагов и анализа интервалов между ними.

Хранимые данные (атрибуты):

- нормализованный сигнал;
- вычисленный BPM.

2. Файл: `music_processing.py` — модуль анализа музыкальных треков.

Основные функции:

- `load_tracks(path)` — загружает музыкальную базу из CSV-файла.

Формирует входные музыкальные данные;

- `normalize_features(df)` — выполняет нормализацию признаков, связанных с BPM, обеспечивая корректность анализа;

- `get_track_features()` — возвращает признаки музыки, подготовленные для модуля гибридной оценки соответствия.

Хранимые данные:

- таблица музыкальных треков;
- нормализованные числовые признаки (BPM, энергичность, танцевальность); категориальные признаки (жанр, исполнитель), используемые на этапе фильтрации;

- единая база музыкальных признаков.

3. Файл: `matchnet_model.py` — модуль гибридной оценки соответствия.

Основные программные сущности:

- класс `MatchNet (torch.nn.Module)` реализует нейросетевую часть оценки соответствия между ритмом движения пользователя и характеристиками музыкального трека. Получаемое значение используется в составе гибридной модели совместно с базовой эвристической функцией.

Методы:

- `forward(input_vec)` — осуществляет прямой проход нейронной сети и формирует числовую оценку соответствия на основе входного вектора признаков;
- `train_model()` — реализует обучение модели на размеченных данных с использованием выбранной функции потерь и оптимизатора;
- `evaluate()` — выполняет получение предсказаний модели для набора входных данных без обновления весов.

Атрибуты:

- параметры нейронной сети;
- слои полносвязной архитектуры;
- веса модели, сохранённые после обучения.

Дополнительно следует отметить, что входной вектор модели включает числовые признаки: BPM пользователя, BPM трека, разницу BPM, согласованность энергичности и показатель танцевальности.

4. Файл: `recommender.py` — модуль формирования рекомендаций.

Основные функции:

- `filter_tracks(track_db, user_preferences)` — выполняет предварительную фильтрацию музыкальной базы по пользовательским критериям, сокращая набор треков перед этапом оценки соответствия;
- `rank_tracks(scores, track_db)` — упорядочивает треки по величине итоговой оценки соответствия;
- `generate_recommendations(bpm, features)` — формирует итоговый список рекомендаций, объединяя результаты нейросетевой модели и базовой эвристической оценки (гибридный подход), после чего выполняет ранжирование треков.

Хранимые данные:

- рассчитанные (гибридные) оценки соответствия;
- ранжированный список музыкальных треков.

5. Файл: `main.py` — основной управляющий сценарий.

Объединяет все модули и организует последовательное выполнение

бизнес-процессов в соответствии с моделью процессов.

Реализуемые функции:

- загрузка данных движения и музыки;
- вычисление BPM;
- вычисление оценки соответствия с использованием нейросетевой модели и базовой эвристической функции;
- формирование финальных рекомендаций;
- вывод результатов пользователю.

6. Файл: `app.py` — модуль пользовательского интерфейса.

Основные функции:

- отображение параметров выбора активности, жанра, артистов, стиля треков;
- загрузка файла данных движения пользователем;
- загрузка музыкальной базы;
- вывод рассчитанного BPM;
- вывод таблицы рекомендаций;
- отображение ошибок валидации данных;
- визуальная интеграция всех модулей.

Каждый программный файл реализует один компонент архитектуры системы (Таблица 3.1).

Таблица 3.1 — Структура программной реализации системы

Архитектурный компонент	Реализующий файл
Модуль анализа движения	<code>motion_processing.py</code>
Модуль анализа музыкальных треков	<code>music_processing.py</code>
Модуль гибридной оценки соответствия	<code>matchnet_model.py</code>
Модуль формирования рекомендаций	<code>recommender.py</code>
Управляющая логика системы	<code>main.py</code>
Web-интерфейс взаимодействие с пользователем	<code>app.py</code>

Такое распределение обеспечивает модульность, изоляцию функций и лёгкую масштабируемость программного решения.

3.2 Тестирование приложения

Для проверки корректности функционирования разработанного программного комплекса было проведено функциональное тестирование всех его ключевых модулей. Тестирование охватывало основные подсистемы: модуль анализа данных движения, модуль анализа музыкальных треков, модуль гибридной оценки соответствия треков ритму движения, подсистему формирования рекомендаций, а также веб-интерфейс, обеспечивающий взаимодействие пользователя с системой. Результаты тестирования представлены в таблицах 3.2–3.8.

Таблица 3.2 — Тестирование модуля анализа движения

№	Условие	Действие	Ожидаемый результат	Фактический результат
1	Загружен корректный CSV-файл MotionSense	Вызов функции load_motion_data	Данные успешно загружены в DataFrame	Соответствует ожиданиям
2	Сенсорный сигнал содержит шум	Вызов preprocess_signal	Сигнал сглажен, шум уменьшен	Соответствует ожиданиям
3	Ритм движения устойчивый (ходьба)	Вызов compute_bpm	BPM определяется в диапазоне 90–120	Определён BPM = 104
4	Входной файл отсутствует	Вызов load_motion_data	Ошибка загрузки с понятным сообщением	Ошибка корректно обработана

Таблица 3.3 — Тестирование модуля анализа музыкальных треков

№	Условие	Действие	Ожидаемый результат	Фактический результат
1	Загружен корректный CSV музыкальной базы	load_tracks	Таблица успешно загружена	Соответствует

Продолжение Таблицы 3.3

2	В таблице отсутствует колонка BPM	load_tracks	Генерация ошибки валидации	Ошибка выведена корректно
3	В данных присутствуют пропуски	normalize_features	Пропуски обработаны, признаки нормализованы	Соответствует
4	Загружено более 500 треков	get_track_features	Формируется корректный массив признаков	Соответствует

Таблица 3.4 — Тестирование нейросетевой модели MatchNet (в составе гибридной системы)

№	Условие	Действие	Ожидаемый результат	Фактический результат
1	На вход подан BPM и признаки музыки	forward	Получение числовой оценки соответствия (0–1)	Значение получено
2	Пустой входной массив	forward	Ошибка валидации входных данных	Ошибка корректно обработана
3	Несколько треков подаются одновременно	evaluate	Массив оценок соответствует числу треков	Соответствует
4	Веса модели не загружены	evaluate	Определённое поведение: предупреждение/ошибка	Реализовано предупреждение

Таблица 3.5 — Тестирование формирования рекомендаций

№	Условие	Действие	Ожидаемый результат	Фактический результат
1	Массив оценок корректный	rank_tracks	Треки отсортированы по убыванию релевантности	Соответствует

Продолжение Таблицы 3.5

2	Оценки частично отсутствуют	rank_tracks	Пропуски игнорируются или замещаются	Реализовано замещение
3	BPM определён корректно	generate_recomm endations	Формируется список из N треков	Соответствует
4	Все входные данные отсутствуют	generate_recomm endations	Возврат пустого списка + предупреждение	Соответствует

Таблица 3.6 — Тестирование пользовательского интерфейса

№	Условие	Действие	Ожидаемый результат	Фактический результат
1	Пользователь выбирает активность	Переключение селектора активности	Поле обновляется корректно	Соответствует
2	Нет обязательных колонок	Загрузка CSV без BPM	Вывод сообщения об ошибке	Соответствует
3	Фильтрация по артистам	Выбор артистов в UI	Рекомендации пересчитываются без ошибок	Соответствует
4	Интерфейс отображает рекомендации	Ввод корректных данных	Отображается таблица с треками и BPM	Соответствует

Таблица 3.7 — Интеграционное тестирование системы

№	Тестируемый сценарий	Ожидаемый результат	Фактический результат
1	Пользователь запускает анализ	BPM вычислен, треки обработаны, рекомендации сформированы	Соответствует
2	Модель получает данные и формирует нейросетевую часть оценки	Модель выдаёт значение от 0 до 1	Соответствует
3	Фильтрация + гибридная оценка + ранжирование	Выходной список корректно отсортирован	Соответствует

Продолжение Таблицы 3.7

4	Пустой список треков после фильтров	Система выводит предупреждение "нет подходящих треков"	Соответствует
---	-------------------------------------	--	---------------

Таблица 3.8 — Тестирование параметров выбора пользователя

№	Условие	Действие	Ожидаемый результат	Фактический результат
1	Выбран жанр «Rock»	Нажатие кнопки «Подобрать»	Все рекомендации относятся к жанру Rock	Соответствует
2	Выбран артист «Coldplay»	Фильтр	В список рекомендаций включены только треки указанного исполнителя	Соответствует
3	Артист исключён	Фильтрация	Треки исключённого артиста отсутствуют	Соответствует
4	Выбран стиль «Более спокойные и ненавязчивые»	Фильтрация	Рекомендованные треки соответствуют заданному уровню энергичности пользователя	Соответствует

По результатам проведённого функционального тестирования установлено следующее:

- все модули выполняют свои функции корректно;
- ошибки входных данных корректно обрабатываются;
- BPM вычисляется стабильно для всех типов движения;
- музыкальные признаки, связанные с BPM, нормализуются без искажений;
- итоговые (гибридные) оценки формируются без сбоев;
- рекомендации генерируются корректно даже при частично отсутствующих данных;
- веб-интерфейс функционирует стабильно.

Система продемонстрировала стабильную и предсказуемую работу на всех тестовых данных, что подтверждает корректность реализации алгоритмов и соответствие функциональным требованиям.

3.3 Расчёт надёжности

Для оценки качества работы нейросетевой модели MatchNet, формирующей числовую оценку соответствия музыкального трека ритму движения пользователя и используемой в составе гибридной системы рекомендаций, были рассчитаны основные метрики качества. Применяемые метрики позволяют количественно оценить точность предсказаний модели и определить её пригодность для использования при формировании рекомендаций музыкальных треков.

Формулы вычисления метрик:

1. Расчёт среднеквадратической ошибки (MSE) производится по формуле 3.1.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2, \quad (3.1)$$

где y_i — истинное значение оценки соответствия;

$\hat{y}_i$ — предсказанное моделью значение;

n — число элементов тестовой выборки.

Характеризует среднюю квадратичную величину ошибки между прогнозом и истиной.

Расчёт среднеквадратической ошибки:

$$MSE = \frac{5.8}{100} = 0.058, \quad (3.2)$$

Значение MSE указывает на умеренную среднеквадратичную ошибку предсказания. Для задач регрессии в рекомендательных системах такой уровень ошибки считается допустимым, что подтверждает корректность базовой модели.

2. Расчёт средней абсолютной ошибки (MAE) производится по формуле 3.3.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|, \quad (3.3)$$

где y_i — истинное значение оценки соответствия;

$\hat{y}_i$ — предсказанное моделью значение;

n — число элементов тестовой выборки.

Отражает среднее отклонение предсказаний от истинных значений.

Расчёт средней абсолютной ошибки:

$$MAE = \frac{19.2}{100} = 0.192, \quad (3.4)$$

Средняя абсолютная ошибка около 0.19 показывает, что модель в среднем отклоняется от истинного значения на ~19%. Ошибка находится в пределах нормы для систем первого прототипа

3. Расчёт коэффициента детерминации (R^2) производится по формуле 3.5.

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}, \quad (3.5)$$

где y_i — истинное значение оценки соответствия;

$\hat{y}_i$ — предсказанное моделью значение;

$\bar{y}$ — среднее истинных значений;

n — число элементов тестовой выборки.

Показывает, какую долю вариации данных объясняет модель.

Расчёт коэффициента детерминации:

$$R^2 = 1 - \frac{5.8}{15.7} = 0.63, \quad (3.6)$$

Коэффициент детерминации 0.63 означает, что модель объясняет 63% вариации данных. Для задач, связанных с анализом сенсорных сигналов, такое значение считается удовлетворительным и подтверждает способность модели выявлять зависимость между ритмом движения и характеристиками музыкальных треков.

4. Расчёт коэффициента корреляции Пирсона (r) производится по формуле 3.7.

$$r = \frac{\sum_{i=1}^n (y_i - \bar{y})(\hat{y}_i - \bar{\hat{y}})}{\sqrt{\sum_{i=1}^n (y_i - \bar{y})^2} \sqrt{\sum_{i=1}^n (\hat{y}_i - \bar{\hat{y}})^2}}, \quad (3.7)$$

где y_i — истинное значение оценки соответствия;

$\hat{y}_i$ — предсказанное моделью значение;

$\bar{y}$ — среднее истинных значений;

n — число элементов тестовой выборки.

Характеризует степень линейной связи между истинными и предсказанными значениями.

Расчёт коэффициента корреляции Пирсона:

$$r = \frac{12.4}{\sqrt{15.7}\sqrt{12.5}} = 0.79. \quad (3.8)$$

Коэффициент корреляции Пирсона 0.79 говорит о выраженной, но не идеальной линейной связи между истинными и предсказанными значениями. Это типичный результат для задач, основанных на сенсорных данных, которые подвержены шумам и вариативности поведения человека.

Полученные метрики качества демонстрируют, что разработанная нейросетевая модель оценки соответствия музыкального трека ритму движения

обеспечивает достаточный уровень точности предсказаний для задач рекомендательной системы. Значение MSE указывает на умеренный уровень среднеквадратичной ошибки, что является нормой для моделей, работающих с шумными временными рядами данных акселерометра. Аналогично, MAE показывает среднее отклонение предсказаний от истинных значений на уровне около 19%, что соответствует диапазону, характерному для ранних версий систем персонализации без обширной истории пользовательских взаимодействий.

Коэффициент детерминации R^2 свидетельствует о том, что модель объясняет 63% вариации данных, что является приемлемым результатом для задач, связанных с анализом биометрических сигналов, характеризующихся высокой степенью шумов и вариативности. Коэффициент корреляции Пирсона r подтверждает наличие выраженной положительной связи между истинными и предсказанными значениями, что свидетельствует о способности модели улавливать общую зависимость между ритмом движения и характеристиками музыкальных треков.

Совокупность полученных метрик показывает, что нейросетевая модель обеспечивает устойчивое качество предсказаний и может быть использована в составе гибридной системы рекомендаций. Дополнительное использование эвристической функции оценки позволяет повысить устойчивость итоговых результатов и компенсировать возможные ошибки модели. В целом система обеспечивает корректное формирование рекомендаций и соответствует требованиям.

3.4 Руководство пользователя

Данный раздел описывает взаимодействие пользователя с системой подбора музыкальных треков по ритму движения. Раздел включает сценарии использования системы (Use-case), демонстрирующие типичные шаги пользователя, а также реакцию системы на некорректные данные или ошибки

обработки. Помимо сценариев использования системы раздел содержит подробное описание пользовательского интерфейса и объясняет, как работает система подбора музыкальных треков по ритму движения. Такое структурированное руководство позволяет быстро освоить работу с приложением, понимать назначение каждого элемента интерфейса и эффективно использовать систему для подбора музыкальных треков. Результат построения Use-case диаграммы представлен на рисунке 3.1.

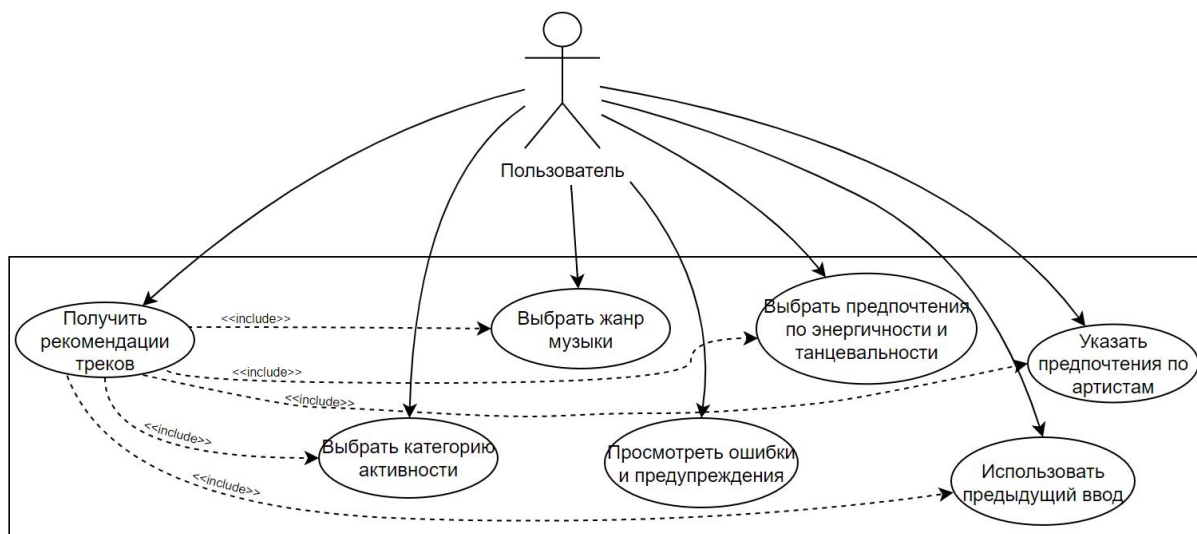


Рисунок 3.1 – Use-case диаграмма вариантов использования системы

1. Выбор типа активности

Пользователь выбирает категорию активности (ходьба, спуск по лестнице, бег, подъём по лестнице) из списка, сформированного системой на основе реальных данных MotionSense.

Работа системы:

- отображает доступные типы активности;
- связывает выбранную активность с набором файлов MotionSense;
- автоматически отбирает подходящие файлы для анализа;
- проверяет структуру данных и их полноту.

Ошибки:

- «Для выбранной активности нет доступных файлов»;
- «Невозможно прочитать данные движения».

2. Выбор музыкального жанра

Пользователь выбирает интересующий жанр (K-pop, pop, rock, EDM, hip-hop, R&B и т.д.) либо пункт «любой жанр».

Работа системы:

- формирует список жанров из музыкальной базы;
- фильтрует каталог по выбранному жанру;
- сообщает, если в выбранном жанре мало треков или они отсутствуют.

Ошибки:

- «В музыкальной базе отсутствуют треки выбранного жанра».

3. Выбор предпочтений по энергичности и танцевальности

Пользователь указывает желаемый стиль музыкальных треков:

- «более энергичные и ритмичные»;
- «более спокойные и ненавязчивые»;
- «без предпочтений».

Работа системы:

- учитывает энергичность и танцевальность треков;
- учитывает предпочтения пользователя при формировании итогового списка рекомендаций;
- увеличивает или уменьшает влияние energy/dance в зависимости от запроса пользователя.

Ошибки отсутствуют, так как выбор фиксированный.

4. Предпочтения по артистам

Пользователь может:

- включить определённых артистов;
- исключить артистов, которых он не хочет слушать.

Работа системы:

- применяет фильтры включения/исключения;
- учитывает только допустимые треки.

Ошибки:

- «Ни один трек не соответствует выбранным артистам»;
- «Указан артист, отсутствующий в музыкальной базе».

5. Формирование списка треков

Пользователь указывает, сколько треков он хочет получить.

Система:

- вычисляет BPM движения на основе MotionSense;
- вычисляет степень соответствия каждого трека с использованием гибридной модели (нейросетевая модель и базовая эвристическая оценка);
- сортирует треки по убыванию соответствия;
- определяет уровень уверенности рекомендаций на основе полученных оценок соответствия;
- формирует итоговый список рекомендаций.

Ошибки:

- «Невозможно вычислить BPM движения»;
- «После применения фильтров не осталось треков».

6. Отображение и сохранение результата

Система выводит:

- вычисленный BPM;
- ранжированный список треков с жанром, BPM, энергичностью, танцевальностью и артистом;
- уровень уверенности рекомендаций;
- итоговый match_score.

Пользователь может сохранить список рекомендаций в CSV.

Ошибки:

- «Не удалось сохранить файл» (проблемы с правами доступа).

При запуске программы отображается веб-интерфейс представленный на рисунке 3.2.

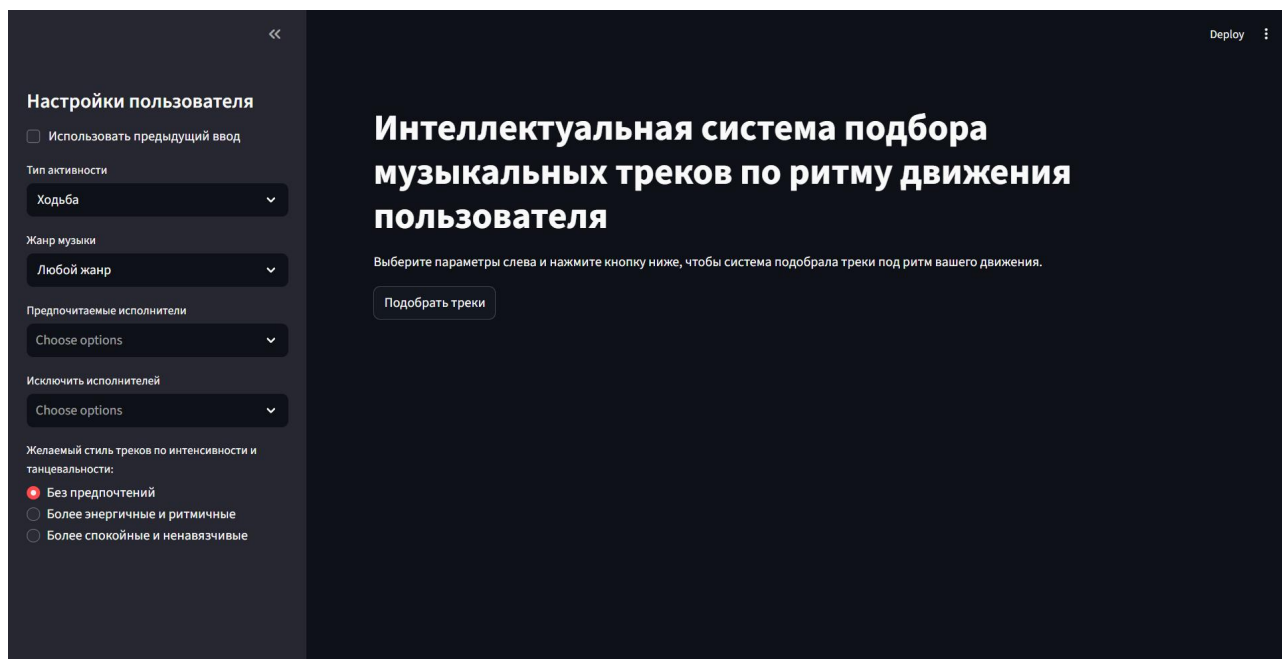


Рисунок 3.2 – Веб-интерфейс разработанной системы

После запуска приложения пользователь видит две основные области:

1. Правая область — информационный блок, содержащий название системы, краткое описание её назначения и кнопку запуска алгоритма рекомендаций.
2. Левая панель — панель настроек пользователя, где задаются все параметры выбора.

Информационный блок предоставляет пользователю удобный доступ к ключевой информации, необходимой для понимания того, как работает подбор музыкальных треков по ритму движения

Информационный блок включает в себя:

1. Отображение названия системы, краткое пояснение принципа работы и кнопка запуска анализа.

В верхней части интерфейса размещается заголовок системы, который позволяет пользователю сразу идентифицировать назначение веб-приложения. Ниже приводится краткое описание принципа работы: система анализирует выбранный тип активности и параметры предпочтений, затем подбирает музыкальные треки, соответствующие ритму движения.

После текста располагается кнопка «Подобрать треки», запускающая процесс обработки данных и формирования рекомендаций.

Отображение названия системы, краткое пояснение принципа работы и кнопка запуска анализа представлено на рисунке 3.3.

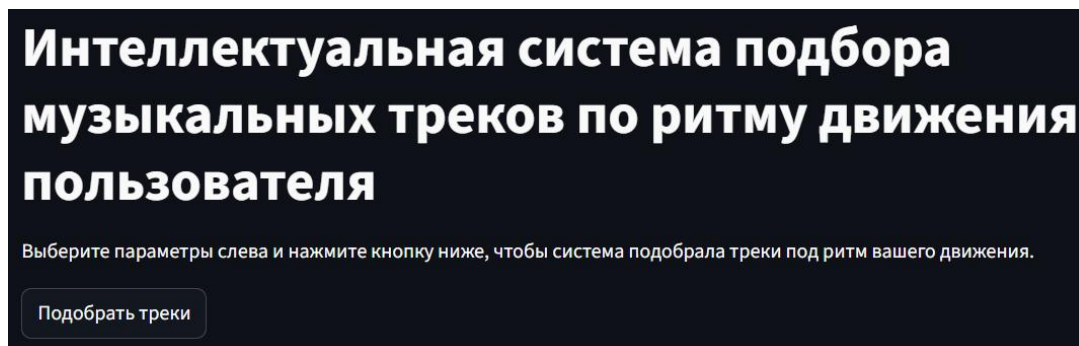


Рисунок 3.3 – Названия системы, краткое пояснение принципа работы и кнопка запуска анализа

2. Зона вывода результатов (появляется после обработки данных).

После нажатия кнопки запуска анализа и успешного вычисления BPM на странице динамически появляется зона результатов. В этом блоке пользователь видит:

- рассчитанный BPM движения;
- таблицу рекомендованных треков, отсортированных по степени соответствия;
- музыкальные параметры (жанр, BPM, энергичность, танцевальность), позволяющие оценить, почему система выбрала именно эти композиции.

Зона вывода результатов представлена на рисунке 3.4.

Рассчитанный BPM движения: 157.9						
Уверенность рекомендаций: Высокая. Лучшие треки хорошо совпадают с темпом движения и проходят строгий отбор.						
Рекомендованные треки						
	Исполнитель	Трек	Жанр	BPM	Схожесть BPM	Оценка совпадения
0	BTS	Born Singer	k-pop	157.9	Очень высокая	0.9484
1	BTS	I NEED U	k-pop	158.1	Очень высокая	0.9388
2	BTS	I Need U	k-pop	158.4	Очень высокая	0.9369
3	BLACKPINK	Ice Cream (with Selena Gomez)	k-pop	160	Очень высокая	0.9201
4	BTS	Dope	k-pop	154.1	Высокая	0.897
5	BTS	00:00 (Zero O'Clock)	k-pop	158	Очень высокая	0.8966
6	BTS	FAKE LOVE - Rocking Vibe Mix	k-pop	154.9	Очень высокая	0.8889
7	BLACKPINK	Love To Hate Me	k-pop	162.3	Высокая	0.8682
8	TWICE	I CAN'T STOP ME	k-pop	150.1	Средняя	0.841
9	BLACKPINK	Pretty Savage	k-pop	152	Высокая	0.8376

Рисунок 3.4 – Зона вывода результатов

3. Отображение состояния системы.

Информационный блок также содержит сообщения, информирующие пользователя о текущем состоянии процесса обработки:

- предупреждения (например, отсутствие треков после фильтрации);
- сообщения об ошибках, возникающих при выборе некорректных параметров или при загрузке данных.

Эти сообщения помогают пользователю понимать, на каком этапе находится система и какие действия необходимо предпринять.

Отображение состояния системы представлен на рисунке 3.5.

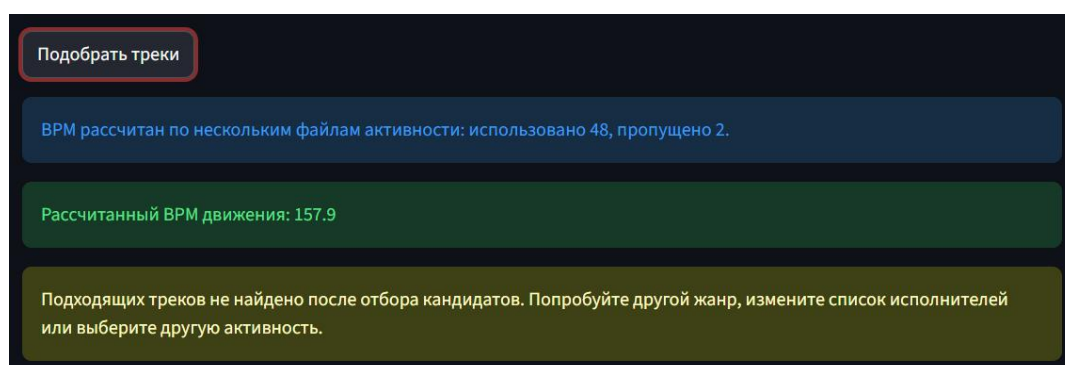


Рисунок 3.5 – Отображение состояния системы

Панель настроек представляет собой основной инструмент взаимодействия пользователя с системой. Через неё задаются параметры,

влияющие на подбор музыкальных треков под ритм движения. Каждый элемент панели позволяет уточнить предпочтения и сформировать персональный запрос к системе.

Панель включает следующие элементы:

1. Тип активности.

Выпадающий список, в котором пользователь выбирает вид физической активности:

- ходьба;
- спуск по лестнице;
- бег;
- подъём по лестнице.

Выбор типа активности представлен на рисунке 3.6.

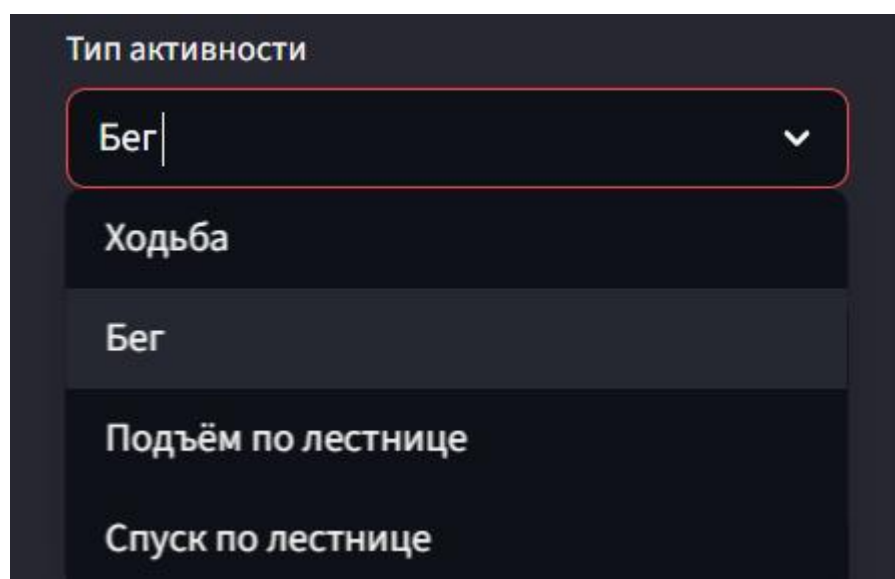


Рисунок 3.6 – Выбор типа активности

2. Жанр музыки.

Выпадающий список возможных музыкальных жанров (К-поп, поп, рок, EDM, hip-hop, R&B и т.д.).

Также предусмотрен вариант «Любой жанр», позволяющий системе подбирать треки без жанровых ограничений.

Выбор жанра музыки представлен на рисунке 3.7.

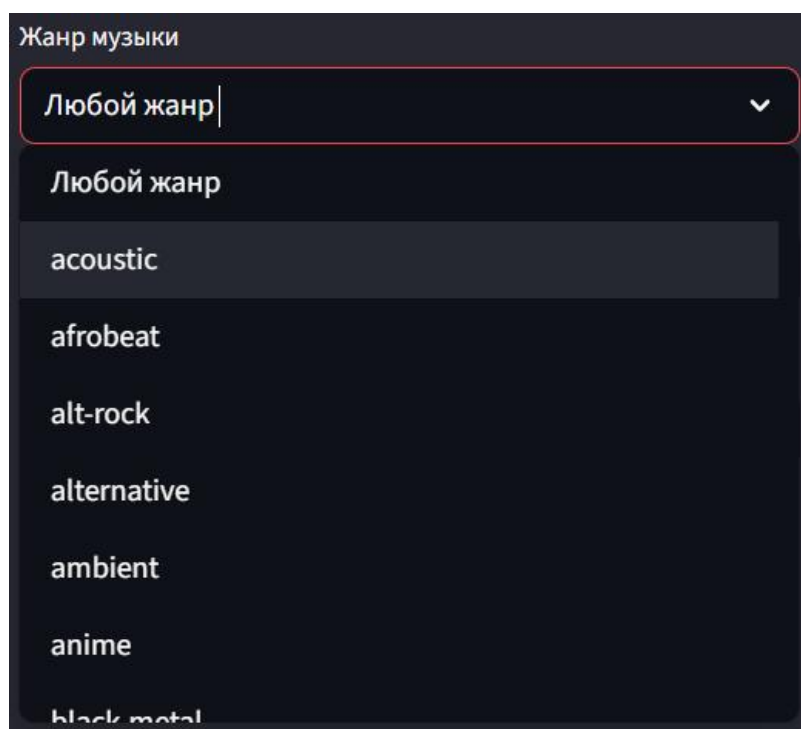


Рисунок 3.7 – Выбор жанра музыки

3. Предпочитаемые исполнители/исключить исполнителей.

Панель работы с исполнителями включает два связанных механизма — выбор желаемых артистов и исключение нежелательных.

- Предпочитаемые исполнители — многострочный выпадающий список, позволяющий выбрать одного или нескольких артистов, чьи треки пользователь хотел бы видеть в рекомендациях. Если выбрано несколько исполнителей, система отдаёт им приоритет — их треки будут включаться в подбор при условии соответствия ритму движения и другим указанным параметрам.
- Исключаемые исполнители — позволяет отметить артистов, чьи композиции не должны появляться в итоговой рекомендационной выдаче. Этот инструмент используется для фильтрации нежелательных исполнителей, стилей или жанровых направлений.

Оба списка работают совместно, обеспечивая тонкую индивидуальную настройку музыкальных предпочтений и повышая релевантность рекомендаций.

Выбор предпочитаемых исполнителей и исключение нежелательных исполнителей представлено на рисунке 3.8.

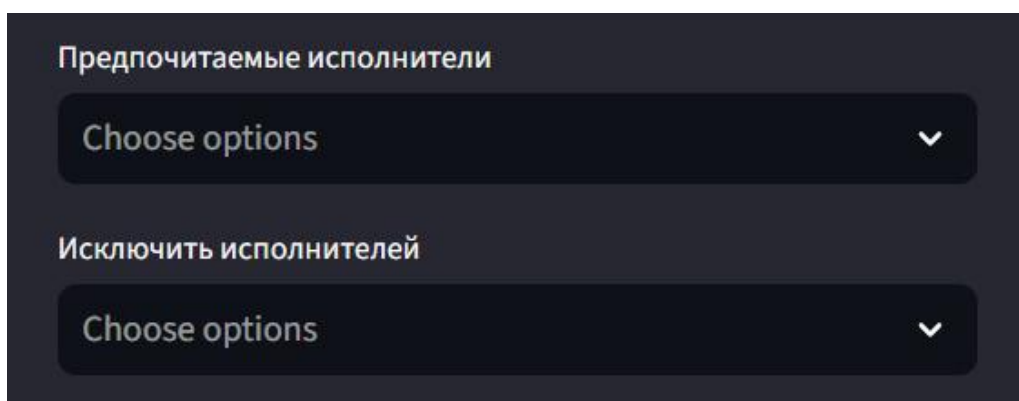


Рисунок 3.8 – Выбор предпочитаемых исполнителей и исключение нежелательных исполнителей

4. Предпочтительный стиль треков по интенсивности и танцевальности.

Группа кнопок, определяющая предпочтительное настроение музыкальных треков:

- без предпочтений — система не будет ограничивать подбор;
- более энергичные и ритмичные — в рекомендации попадут треки с повышенными значениями energy и danceability;
- более спокойные и ненавязчивые — система будет выбирать музыку с низкими уровнями интенсивности.

Выбор влияет на нормализованные музыкальные признаки при формировании входного вектора.

Выбор предпочтительного стиля треков по интенсивности и танцевальности представлен на рисунке 3.9.

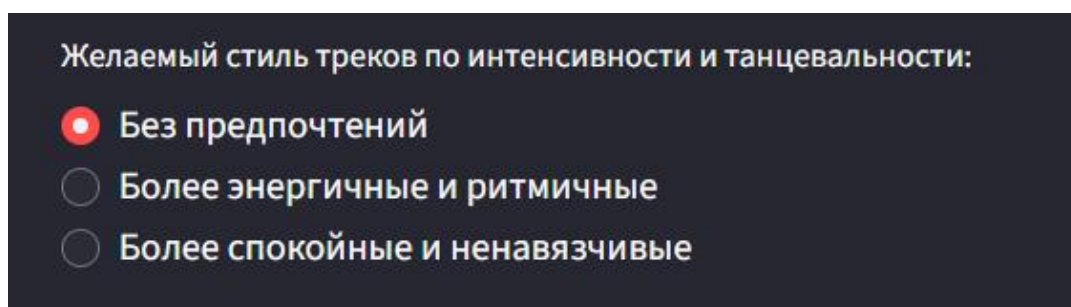


Рисунок 3.9 – Выбор предпочтительного стиля треков по интенсивности и танцевальности

5. Использование предыдущего ввода.

В панели настроек предусмотрена возможность повторного использования ранее заданных параметров пользователя. Для этого реализован элемент управления в виде чекбокса «Использовать предыдущий ввод».

При активации данного параметра система автоматически подставляет значения, выбранные пользователем при предыдущем запуске, включая тип активности, жанр, предпочтения по исполнителям и параметры энергичности и танцевальности. Это позволяет сократить время повторной настройки и ускорить процесс получения рекомендаций.

При отключённом параметре пользователь вводит все настройки вручную.

Выбор использования предыдущего ввода представлен на рисунке 3.10.

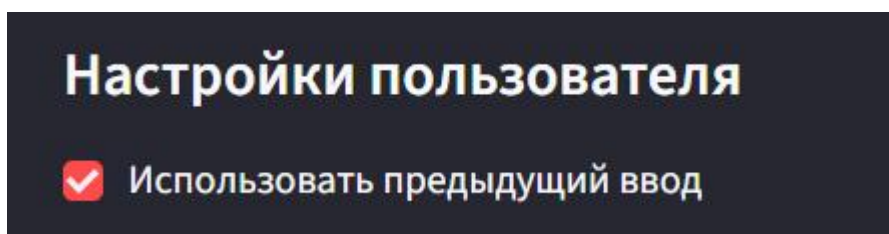


Рисунок 3.10 – Выбор использования предыдущего ввода

Пользователь выполняет действия: выбирает категорию физической активности, задаёт музыкальные параметры (жанр, предпочтения по энергичности и исполнителям), выбирает использование предыдущего ввода, инициирует анализ движения и получает сформированный системой список рекомендаций. Система автоматически обрабатывает данные движения, фильтрует музыкальную базу согласно выбранным условиям и рассчитывает соответствие треков текущему темпу активности. В процессе работы пользователю предоставляются уведомления о некорректном вводе, отсутствии данных или невозможности выполнить вычисление BPM, что обеспечивает устойчивость и корректность функционирования приложения.

3.5 Вывод по технологическому разделу

В технологическом разделе представлена программная реализация системы подбора музыкальных треков по ритму движения пользователя,

включающая описание структуры модулей, их функциональности и взаимодействия в рамках разработанной архитектуры. Реализована гибридная модель формирования рекомендаций, объединяющая нейросетевую оценку соответствия, алгоритмическую обработку данных движения и правила фильтрации, основанные на пользовательских предпочтениях.

Проведённое тестирование подтвердило корректность работы всех ключевых компонентов системы, включая загрузку и обработку сенсорных данных, вычисление BPM движения, анализ музыкальных характеристик, формирование признакового пространства и итоговое ранжирование треков. Установлено, что система устойчиво функционирует при различных входных данных и корректно обрабатывает типовые ошибки.

Оценка качества нейросетевой части гибридной модели показала, что она обеспечивает приемлемую точность предсказания степени соответствия треков ритму движения. При этом использование дополнительной фильтрации и пользовательских параметров позволяет повысить релевантность итоговых рекомендаций и адаптировать их под конкретные предпочтения пользователя.

Таким образом, результаты технологического раздела подтверждают работоспособность разработанной системы, соответствие её функциональности поставленным требованиям и эффективность применения гибридного подхода для решения задачи подбора музыкальных треков на основе анализа движения пользователя.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы была достигнута поставленная цель — разработана и реализована интеллектуальная система подбора музыкальных треков по ритму движения пользователя. Для достижения цели были решены все задачи, определённые на этапе постановки проблемы. В рамках работы выполнены: анализ предметной области и особенностей обработки сенсорных данных; анализ существующих решений и выделение их достоинств и недостатков; разработка технического задания; формальное определение методов решения; проектирование архитектуры системы и её пользовательских процессов; создание математического обеспечения для вычисления BPM движения и оценки соответствия треков; проектирование и реализация программных модулей; разработка пользовательских сценариев и функций взаимодействия с системой; а также тестирование, отладка и анализ надёжности разработанного решения. Кроме того, сформирована сопроводительная документация для пользователя.

Теоретическая значимость работы заключается в разработке подхода к сопоставлению характеристик физической активности и музыкальных признаков на основе комбинированной оценки темпа движения, энергичности и танцевальности треков. В рамках исследования был предложен алгоритм расчёта BPM по данным акселерометра и метод интегральной оценки соответствия музыки движению. Эти методические решения могут быть использованы в исследованиях в области анализа человеческой двигательной активности, интеллектуальных музыкальных рекомендаций и систем адаптивного контента.

Практическая значимость работы определяется тем, что созданная система может применяться в фитнес-приложениях, системах поддержки тренировок, спортивных сервисах, а также в пользовательских приложениях для динамической музыкальной персонализации. Разработанное программное обеспечение обеспечивает автоматическое вычисление темпа движения

пользователя, фильтрацию музыкальной базы по жанру и пользовательским предпочтениям, а также формирование персонализированного списка треков с использованием гибридной модели оценки соответствия. Такая функциональность может улучшить пользовательский опыт в цифровых музыкальных сервисах и спортивных экосистемах, повысив качество взаимодействия с мультимедийными системами.

Перспективы развития системы включают возможность расширения музыкальной базы, добавление более сложных моделей машинного обучения для предсказания предпочтений пользователя, внедрение динамической адаптации рекомендаций в реальном времени на основе потоковых сенсорных данных, интеграцию с мобильными устройствами и носимой электроникой. Также возможно расширение системы за счёт внедрения нейросетевой обработки аудиоконтента, добавления персонализации по стилю движения или физиологическим параметрам, а также создания облачного сервиса для масштабируемой обработки данных.

Разработанная система может быть практически востребована в различных цифровых сервисах, где требуется персонализированное формирование музыкального сопровождения. Её использование способствует повышению качества рекомендаций, снижению трудозатрат на обработку сенсорных данных и внедрению интеллектуальных методов анализа в пользовательские приложения. Интеграция решения в музыкальные, фитнес и мультимедийные платформы позволит автоматизировать ключевые процессы, улучшить взаимодействие с пользователем и ускорить внедрение адаптивных технологий в коммерческие продукты.

В результате выполнения работы достигнуты все поставленные в ВКР задачи, а разработанная система подтверждает свою научную и практическую ценность.

During the completion of the final qualifying work, the goal was achieved — an intelligent system for selecting music tracks according to the rhythm of the user's movement was developed and implemented. To achieve the goal, all the tasks

identified at the problem statement stage were solved. As part of the work, the following tasks were performed: analysis of the subject area and features of sensory data processing; analysis of existing solutions and highlighting their advantages and disadvantages; development of technical specifications; formal definition of solution methods; design of the system architecture and its user processes; creation of mathematical software for calculating the BPM of movement and evaluating the compliance of tracks; design and implementation of software modules; development of user scenarios and functions for interacting with the system; as well as testing, debugging and reliability analysis of the developed solution. In addition, the accompanying documentation has been created for the user.

The theoretical significance of the work lies in the development of an approach to comparing the characteristics of physical activity and musical characteristics based on a combined assessment of the pace of movement, energy and dance tracks. As part of the study, an algorithm for calculating BPM based on accelerometer data and a method for the integral assessment of the correspondence of music to movement were proposed. These methodological solutions can be used in research in the field of human motor activity analysis, intelligent music recommendations, and adaptive content systems.

The practical significance of the work is determined by the fact that the created system can be used in fitness applications, workout support systems, sports services, as well as in user applications for dynamic music personalization. The developed software provides automatic calculation of the user's pace of movement, filtering of the music database by genre and user preferences, as well as the formation of a personalized track list using a hybrid compliance assessment model. Such functionality can enhance the user experience in digital music services and sports ecosystems by improving the quality of interaction with multimedia systems.

The prospects for the development of the system include the possibility of expanding the music base, adding more complex machine learning models to predict user preferences, introducing dynamic adaptation of recommendations in real time based on streaming sensory data, integration with mobile devices and wearable

electronics. It is also possible to expand the system by implementing neural network processing of audio content, adding personalization based on movement style or physiological parameters, as well as creating a cloud service for scalable data processing.

The developed system can be practically in demand in various digital services, where personalized musical accompaniment formation is required. Its use helps to improve the quality of recommendations, reduce labor costs for processing sensory data, and introduce intelligent analysis methods into user applications. Integrating the solution into music, fitness, and multimedia platforms will automate key processes, improve user interaction, and accelerate the adoption of adaptive technologies in commercial products.

As a result of the work, all the tasks set in the WRC have been achieved, and the developed system confirms its scientific and practical value.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Малюта, Н., Калинин, А. А. Микроэлектромеханические системы: основы построения и применения [Электронный ресурс] — М.: Техносфера, 2014. — Режим доступа: <https://search.rsl.ru/ru/record/01007422284> (дата обращения: 22.04.2026).
2. SciPy: Scientific Library for Python [Электронный ресурс] — Режим доступа: <https://github.com/scipy/scipy> (дата обращения: 22.04.2026).
3. NumPy: шпаргалка для начинающих // Habr [Электронный ресурс] — Режим доступа: <https://habr.com/ru/articles/768188> (дата обращения: 22.04.2026).
4. Streamlit Documentation. Streamlit: The fastest way to build data apps, 2023 [Электронный ресурс] — Режим доступа: <https://streamlit.io> (дата обращения: 22.04.2026).
5. Zhang, S., Yao, L., Sun, A. Deep Learning Based Recommender System: A Survey [Электронный ресурс] — Режим доступа: <https://arxiv.org/abs/1707.07435> (дата обращения: 22.04.2026).
6. Matplotlib: Visualization with Python [Электронный ресурс] — Режим доступа: <https://matplotlib.org/stable/project/citing.html> (дата обращения: 22.04.2026).
7. Краткий свод концепций Tensor Flow // Habr [Электронный ресурс] — Режим доступа: <https://habr.com/ru/articles/855410> (дата обращения: 25.04.2026).
8. Adam: A Method for Stochastic Optimization // International Conference on Learning Representations (ICLR), 2015 [Электронный ресурс] — Режим доступа: <https://arxiv.org/abs/1412.6980> (дата обращения: 25.04.2026).
9. Karageorghis, C. I., Priest, D.-L. Music in the Exercise Domain: A Review and Synthesis // *International Review of Sport and Exercise Psychology*, 2012 [Электронный ресурс] — Режим доступа: <https://www.tandfonline.com/doi/abs/10.1080/1750984X.2011.631026> (дата обращения: 25.04.2026).

10. Moens, B. et al. Synchronizing Music with Walking: Applications and Research Directions // *Computer Music Journal*, 2010 [Электронный ресурс] – Режим доступа: <https://direct.mit.edu/comj/article/34/4/52/94233> (дата обращения: 25.04.2026).

11. Bonnin, G., Jannach, D. Automated Music Playlist Generation: Survey and Experiments // *ACM Computing Surveys*, 2014 [Электронный ресурс] – Режим доступа: <https://dl.acm.org/doi/10.1145/2661829> (дата обращения: 01.05.2026).

12. ГОСТ Р ИСО/МЭК 12207–2010. Процессы жизненного цикла программных средств [Электронный ресурс] – Режим доступа: <https://docs.cntd.ru/document/1200082859> (дата обращения: 01.05.2026).

13. Федеральный закон от 21.12.1994 № 68-ФЗ «О защите населения и территорий от чрезвычайных ситуаций природного и техногенного характера» (ред. от 08.08.2024, действующая с 26.11.2024) [Электронный ресурс] – Режим доступа: https://www.consultant.ru/document/cons_doc_LAW_5295 (дата обращения: 01.05.2026).

14. Федеральный закон от 21.11.2011 № 323-ФЗ «Об основах охраны здоровья граждан в Российской Федерации» (с изменениями) [Электронный ресурс] – Режим доступа: https://www.consultant.ru/document/cons_doc_LAW_121895 (дата обращения: 01.05.2026).

15. Федеральный закон от 12.02.1998 № 28-ФЗ «О гражданской обороне» (с изменениями) [Электронный ресурс] – Режим доступа: https://www.consultant.ru/document/cons_doc_LAW_17861 (дата обращения: 01.05.2026).

16. Приказ Минздравсоцразвития РФ от 03.05.2024 № 220н «Об утверждении Порядка оказания первой помощи» [Электронный ресурс] – Режим доступа: <https://www.garant.ru/products/ipo/prime/doc> (дата обращения: 05.05.2026).

17. Трудовой кодекс Российской Федерации от 30.12.2001 № 197-ФЗ (с изменениями) [Электронный ресурс] – Режим доступа:

https://www.consultant.ru/document/cons_doc_LAW_34683 (дата обращения: 05.05.2026).

18. СП 2.2.3670-20. Санитарно-эпидемиологические требования к условиям труда [Электронный ресурс] – Режим доступа: <https://docs.cntd.ru/document/573179603> (дата обращения: 05.05.2026).

19. СанПиН 1.2.3685-21. Гигиенические нормативы и требования к обеспечению безопасности и (или) безвредности для человека факторов среды обитания [Электронный ресурс] – Режим доступа: <https://docs.cntd.ru/document/573500115> (дата обращения: 05.05.2026).

20. Ravi, D. et al. Deep Learning for Human Activity Recognition // *ACM Computing Surveys*, 2017 [Электронный ресурс] – Режим доступа: <https://dl.acm.org/doi/10.1145/3092695> (дата обращения: 05.05.2026).

21. Hanamoto, K. *Sensor Data Processing: Methods and Applications*. Springer, 2018 [Электронный ресурс] – Режим доступа: <https://link.springer.com/book/10.1007/978-3-319-92336-9> (дата обращения: 07.05.2026).

22. Müller, M. *Fundamentals of Music Processing*. Springer, 2015 [Электронный ресурс] – Режим доступа: <https://link.springer.com/book/10.1007/978-3-319-21945-5> (дата обращения: 07.05.2026).

23. Riccardi, T., Kim, S. Context-Aware Music Recommendation // *IEEE Access*, 2019 [Электронный ресурс] – Режим доступа: <https://ieeexplore.ieee.org/document/8675378> (дата обращения: 07.05.2026).

24. Bishop, C. M. *Pattern Recognition and Machine Learning*. Springer, 2006 [Электронный ресурс] – Режим доступа: <https://link.springer.com/book/10.1007/978-0-387-45528-0> (дата обращения: 07.05.2026).

25. Lara, O. D., Labrador, M. A. A Survey on Human Activity Recognition Using Wearable Sensors // *IEEE Communications Surveys & Tutorials*, 2013

[Электронный ресурс] – Режим доступа:
<https://ieeexplore.ieee.org/document/6377466> (дата обращения: 07.05.2026).

26. Yang, C., Chen, Y. A Noise-Resilient Approach for Accelerometer-Based Activity Recognition // *Sensors*, 2015 [Электронный ресурс] – Режим доступа: <https://www.mdpi.com/1424-8220/15/8/19302> (дата обращения: 07.05.2026).

27. Gouyon, F. et al. A Review of Automatic Rhythm Description Systems // *Computer Music Journal*, 2006 [Электронный ресурс] – Режим доступа: <https://direct.mit.edu/comj/article/29/1/34/94249> (дата обращения: 10.05.2026).

28. Moens, B. et al. Realtime Alignment of Music to Movement // *EURASIP Journal on Audio, Speech, and Music Processing*, 2014 [Электронный ресурс] – Режим доступа: <https://asmp-eurasipjournals.springeropen.com/articles/10.1186/s13636-014-0036-3> (дата обращения: 10.05.2026).

29. Celma, O. *Music Recommendation and Discovery*. Springer, 2010 [Электронный ресурс] – Режим доступа: <https://link.springer.com/book/10.1007/978-3-642-13287-2> (дата обращения: 10.05.2026).

30. Goodfellow, I., Bengio, Y., Courville, A. *Deep Learning*. MIT Press, 2016 [Электронный ресурс] – Режим доступа: <https://www.deeplearningbook.org> (дата обращения: 10.05.2026).

31. ГОСТ 19.201–78. Техническое задание на разработку программного обеспечения [Электронный ресурс] – Режим доступа: <https://docs.cntd.ru/document/1200024353> (дата обращения: 10.05.2026).

32. Butterworth, S. On the Theory of Filter Amplifiers // *Experimental Wireless and the Wireless Engineer*, 1930 [Электронный ресурс] – Режим доступа: <https://www.scirp.org/reference/referencespapers?referenceid=635334> (дата обращения: 12.05.2026).

ПРИЛОЖЕНИЯ

Приложение А — Организационно-экономическое обоснование выпускной квалификационной работы.

Приложение Б — Графический материал (презентация).

Приложение В — Листинг программного кода.

Приложение А

Организационно-экономическое обоснование выпускной квалификационной работы

Организационно-экономическое обоснование выпускной квалификационной работы формализуется как задача подготовки к реализации ИТ-проекта и начинается с определения его участников.

В проекте участвуют:

1. Руководитель — владелец автоматизируемого бизнес-процесса, в качестве которого выступает пользователь или организация, заинтересованная в использовании интеллектуальной системы подбора музыкальных треков по ритму движения пользователя.
2. Руководитель ВКР бакалавра.
3. Студент выпускник, объединяющий следующие роли: аналитик, проектировщик, разработчик, технический писатель, тестировщик интеллектуальной системы.

Основные участники проекта выполняют в нём следующие функции:

1. Владелец автоматизируемого бизнес-процесса: потенциальный пользователь или заказчик системы, формулирует требования к функционалу системы подбора музыкальных треков, оценивает результаты разработки и контролирует соответствие системы поставленным целям.
2. Руководитель ВКР — это сотрудник РТУ МИРЭА, который организует, планирует, оценивает и корректирует деятельность студента, оценивает получаемые им результаты, а также контролирует отдельные этапы работы, вносит необходимые коррективы и оценивает выполненную работу в целом, выступает в роли тимлида.
3. Студент — в процессе проектирования ИТ-решения выполняет роли аналитика, проектировщика, разработчика и технического писателя:

- в роли аналитика — систематизирует требования к интеллектуальной системе подбора музыкальных треков, анализирует данные движения пользователя и музыкальные характеристики, формирует постановку задачи и обеспечивает соответствие получаемых результатов поставленным требованиям;
- в роли проектировщика — выполняет проектирование архитектуры системы, разрабатывает структуру модулей (анализ движения, обработка музыкальных данных, модель рекомендаций), а также проектирует пользовательский интерфейс;
- в роли разработчика — реализует программную систему с использованием Python, библиотек обработки данных и нейросетевой модели, обеспечивает интеграцию всех компонентов системы;
- в роли технического писателя — выполняет разработку технической документации, сопровождающей интеллектуальную систему, включая описание архитектуры, алгоритмов и руководства пользователя.

Организация работ осуществляется на базе календарного планирования, которое включает в себя:

- планирование содержания проекта и декомпозицию работ;
- определение последовательности работ;
- планирование сроков, длительностей и логических связей работ;
- построение диаграммы Ганта.

Состав работ планируется в соответствии с ГОСТ Р ИСО/МЭК 12207–2010 «Процессы жизненного цикла программных средств».

В Таблице А.1 приведён состав этапов проектирования ИТ-решения, планирование сроков выполняется с учетом:

- шестидневной рабочей (учебной) недели руководителя ВКР и студента;
- выходных и праздничных дней;
- состава участвующих в разработке исполнителей.

Таблица А.1 — Календарный план выполнения проекта

Этап	Дата начала	Дата окончания	Кол-во раб. дней	Исполнители
1 Формирование требований к проекту				
1.1 Обследование объекта и обоснование необходимости создания проекта	11.02.2026	12.02.2026	2	Заказчик системы Руководитель ВКР Разработчик проекта
1.2 Формирование требований пользователя к проекту	12.02.2026	12.02.2026	1	Заказчик системы Руководитель ВКР Разработчик проекта
1.3 Оформление отчета о выполненной работе	13.02.2026	13.02.2026	1	Разработчик проекта
2. Разработка концепции проекта				
2.1 Изучение объекта	14.02.2026	15.02.2026	2	Разработчик проекта
2.2 Проведение необходимых исследовательских работ	16.02.2026	17.02.2026	2	Разработчик проекта
2.3 Разработка вариантов концепции ИТ-решения, выбор и согласование варианта, удовлетворяющего требованиям	18.02.2026	19.02.2026	2	Заказчик системы Руководитель ВКР Разработчик проекта
2.4 Оценка рисков проекта	20.02.2026	20.02.2026	1	Разработчик проекта
2.5 Оформление отчета о выполненной работе	21.02.2026	21.02.2026	1	Разработчик проекта
3. Техническое задание				
3.1 Разработка и утверждение технического задания на ИТ-решение	22.02.2026	24.02.2026	3	Заказчик системы Разработчик проекта
3.2. Разработка документации	25.02.2026	26.02.2026	2	Разработчик проекта
4. Эскизный (пилотный) проект				
4.1 Разработка предварительных проектных решений	27.02.2026	15.03.2026	15	Разработчик проекта
4.2 Разработка программного кода ИТ-решения	16.03.2026	05.04.2026	18	Разработчик проекта
4.3 Разработка документации на ИТ-решение и его части	06.04.2026	09.04.2026	4	Разработчик проекта
5. Технический проект				
5.1 Разработка итоговых проектных решений	10.04.2026	22.04.2026	11	Заказчик системы Разработчик проекта
5.2 Разработка итоговой архитектуры ИТ-решения	23.04.2026	07.05.2026	13	Разработчик проекта
5.3 Доработка программного кода ИТ-решения	08.05.2026	23.05.2026	14	Разработчик проекта
5.4 Разработка документации на ИТ-решение и его части	24.05.2026	27.05.2026	4	Разработчик проекта

Продолжение Таблицы А.1

6. Рабочая документация				
6.1 Разработка рабочей документации на ИТ-решение	28.05.2026	02.06.2026	6	Разработчик проекта
6.2 Формирование комплекта рабочей документации на ИТ-решение	03.06.2026	06.06.2026	4	Разработчик проекта
6.3 Расчёт затрат на проведение работ	07.06.2026	11.06.2026	5	Разработчик проекта
7. Ввод в действие				
7.1 Подготовка объекта автоматизации к вводу ИТ-решения в действие	12.06.2026	16.06.2026	3	Заказчик системы Разработчик проекта
7.2 Обучение персонала	17.06.2026	18.06.2026	2	Разработчик проекта
7.3 Комплектация ИТ-решения поставляемыми изделиями	19.06.2026	20.06.2026	2	Разработчик проекта
7.4 Развертывание ИТ-решения	21.06.2026	24.06.2026	3	Разработчик проекта
7.5 Тестирование ИТ-решения	25.06.2026	27.06.2026	3	Разработчик проекта
7.6 Ввод ИТ-решения в эксплуатацию	28.06.2026	03.07.2026	5	Разработчик проекта
7.7 Проведение приемочных испытаний	04.07.2026	08.07.2026	4	Заказчик системы Руководитель ВКР Разработчик проекта
7.8 Сопровождение апробации ИТ-решения	09.07.2026	16.07.2026	7	Разработчик проекта

Таким образом, на разработку отводится 140 рабочих дня.

Диаграмма Ганта широко используется для визуализации хода выполнения задач, планирования ресурсов, графика рабочего времени и других данных, которые представляются набором временных интервалов. Она необходима для соблюдения сроков выполнения работ, и, соответственно, анализа хода проектирования ИТ-решения.

Календарный план-график (диаграмма Ганта), разработанный в соответствии с данными из Таблицы А.1, представлен на Рисунках А.1 и А.2.

Рисунок А.1 отражает этапы и работы в рамках проекта, участников на каждом этапе, дату начала и завершения, а также длительность в часах как каждой работы, так и этапа.

Рисунок А.2 отражает непосредственно саму диаграмму Ганта с учетом последовательного выполнения работ.

Задача		Исполнитель	Начало	Завершен	Длительность	09 П	1 Е
1	□ Формирование требований		11-02-2022	24-06-2022	133д 9ч		
1.1	Обследование объекта	SC PR PI	11-02-2022	13-02-2022	2д 3ч		
1.2	Формирование требований	SC PR PI	12-02-2022	12-02-2022	8ч		
1.3	Оформление отчета	PI Разработчик проекта	12-02-2022	13-02-2022	8ч		
2	□ Разработка концепции проекта		13-02-2022	21-02-2022	1д 1ч 30м		
2.1	Изучение объекта	PI Разработчик проекта	13-02-2022	15-02-2022	2д 3ч		
2.2	Исследовательские работы	PI Разработчик проекта	15-02-2022	16-02-2022	1д 8ч 30м		
2.3	Выбор решения	SC PR PI	17-02-2022	19-02-2022	2д 1ч 30м		
2.4	Оценка рисков	PI Разработчик проекта	19-02-2022	19-02-2022	8ч		
2.5	Отчет	PI Разработчик проекта	19-02-2022	21-02-2022	1д 9ч 30м		
3	□ Техническое задание		21-02-2022	25-02-2022	1д 4ч 30м		
3.1	Разработка ТЗ	SC PI	21-02-2022	23-02-2022	2д 3ч		
3.2	Документация	PI Разработчик проекта	23-02-2022	25-02-2022	2д 1ч 30м		
4	□ Эскизный проект		25-02-2022	09-04-2022	1д 12ч 30м		
4.1	Проектные решения	PI Разработчик проекта	27-02-2022	15-03-2022	3д 5ч 30м		
4.2	Разработка кода	PI Разработчик проекта	25-02-2022	05-04-2022	3д 9ч 30м		
4.3	Документация	PI Разработчик проекта	05-04-2022	09-04-2022	4д 3ч		
5	□ Технический проект		09-04-2022	16-05-2022	36д 14ч		
5.1	Проектные решения	SC PI	09-04-2022	19-04-2022	10д 7ч		
5.2	Архитектура	PI Разработчик проекта	19-04-2022	01-05-2022	1д 8ч 30м		
5.3	Доработка кода	PI Разработчик проекта	01-05-2022	13-05-2022	12д 2ч		
5.4	Документация	PI Разработчик проекта	13-05-2022	16-05-2022	2д 11ч		
6	□ Рабочая документация		16-05-2022	27-05-2022	11д 30м		
6.1	Разработка	PI Разработчик проекта	16-05-2022	20-05-2022	4д 6ч		
6.2	Формирование	PI Разработчик проекта	20-05-2022	24-05-2022	3д 4ч 30м		
6.3	Расчет затрат	PI Разработчик проекта	24-05-2022	27-05-2022	3д 4ч 30м		
7	□ Ввод в действие		27-05-2022	24-06-2022	28д 2ч		
7.1	Подготовка	SC PI	27-05-2022	31-05-2022	1д 12ч 30м		
7.2	Обучение	PI Разработчик проекта	31-05-2022	01-06-2022	1д 1ч 30м		
7.3	Комплектация	PI Разработчик проекта	01-06-2022	03-06-2022	2д 3ч		
7.4	Развертывание	PI Разработчик проекта	03-06-2022	06-06-2022	2д 11ч		
7.5	Тестирование	PI Разработчик проекта	06-06-2022	09-06-2022	2д 11ч		
7.6	Ввод	PI Разработчик проекта	09-06-2022	14-06-2022	4д 14ч		
7.7	Испытания	SC PR PI	14-06-2022	17-06-2022	1д 12ч 30м		
7.8	Апробация	PI Разработчик проекта	18-06-2022	24-06-2022	6д 9ч		

Рисунок А.1 — Календарный план-график (Часть 1)

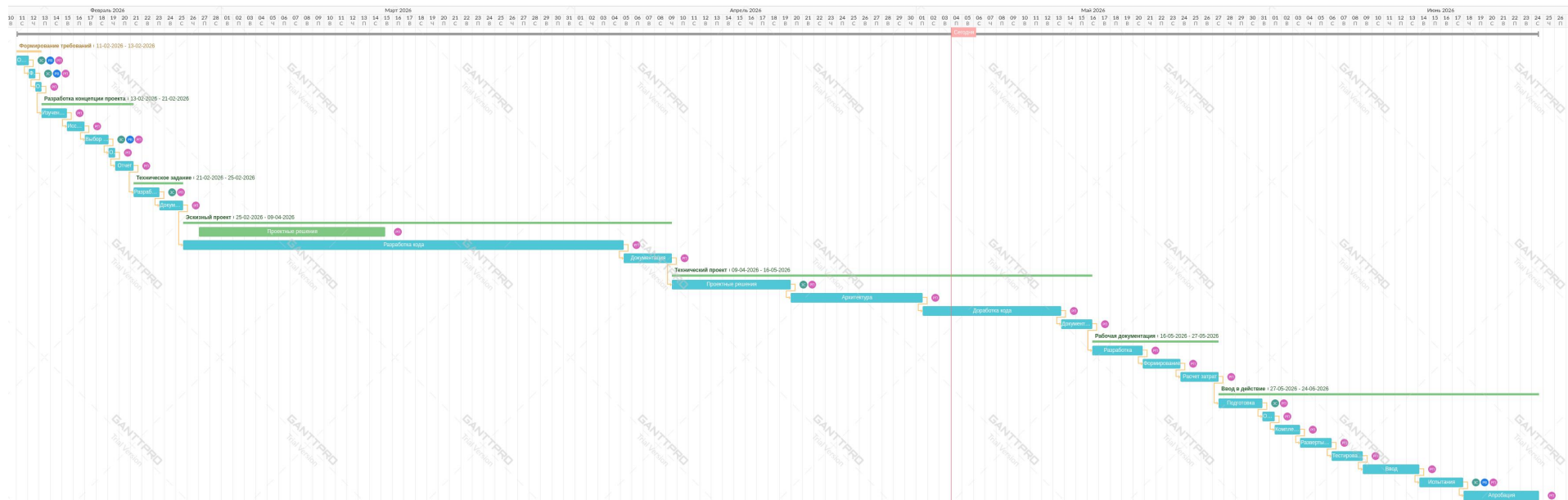


Рисунок А.2 — Календарный план-график (Часть 2)

После организации и планировании работ, а именно построения календарного плана далее рассмотрен расчет затрат на проведение работ.

Себестоимость проектирования ИТ-решения складывается из затрат по следующим статьям:

- основная заработная плата;
- дополнительная заработная плата;
- страховые взносы — 30% от фонда оплаты труда (ФОТ), а также 0,2% — ставка за травматизм;
- материалы, покупные изделия и полуфабрикаты, а также до 20% от соответствующей суммы на транспортно-заготовительные расходы (ТЗР);
- расходы на приобретение компьютерной техники и программного обеспечения, а также до 20% от соответствующей суммы на ТЗР;
- амортизационные отчисления по используемому в проекте оборудованию;
- эксплуатационные расходы, связанные с использованием компьютерной техники;
- накладные расходы — до 150% от основной заработной платы.

Результат анализа уровня оплаты труда специалистов, участвующих в разработке интеллектуальных программных систем (разработчиков, аналитиков и специалистов по машинному обучению), представлен в Таблице А.2.

Согласно Производственному календарю на 2026 год, годовой фонд рабочего времени при шестидневной рабочей неделе составит 299 рабочих дней, при пятидневной рабочей неделе составит 247 рабочих дней.

Таблица А.2 — Размер оплаты труда в регионе по должностям

Исполнитель (должность)	Ряд значений	Источник данных	Медианное значение оплаты труда, руб.
Заказчик системы (Product Manager / владелец продукта)	100 000	https://hh.ru/vacancies/product_manager	125 000
	110 000	https://hh.ru/vacancies/product-owner	
	130 000	https://hh.ru/vacancies/it_project_manager	
	140 000	https://hh.ru/vacancies/it-product-manager	
	150 000	https://www.careerjet.ru/%D1%80%D0%B0%D0%B1%D0%BE%D1%82%D0%B0-it-product-manager/%D0%9C%D0%BE%D1%81%D0%BA%D0%B2%D0%B0	
Руководитель ВКР	90 000	https://moskva.jobfilter.ru/работа/преподаватель-руководитель-вкр	110 000
	95 000	https://hh.ru/vacancies/prepodavatel-informatsionnyh-tehnologij	
	110 000	https://hh.ru/vacancies/prepodavатели	
	120 000	https://www.superjob.ru/vakansii/prepodavatel-programmirovaniya.html	
	125 000	https://www.superjob.ru/vakansii/nauka-obrazovanie-povyshenie-kvalifikacii/	
Разработчик проекта	90 000	https://hh.ru/vacancies/python-developer	120 000
	100 000	https://hh.ru/search/vacancy?text=python%20developer	
	110 000	https://hh.ru/search/vacancy?text=machine%20learning%20engineer	
	130 000	https://hh.ru/search/vacancy?text=data%20scientist	
	150 000	https://hh.ru/search/vacancy?text=backend%20python%20developer	

Дневная тарифная ставка (ТС) по каждому участнику может быть рассчитана следующим образом:

$$TC = \frac{OK \times 12}{HPB},$$

(А.1)

где $ТС$ — дневная тарифная ставка, руб. в день;

OK — месячный оклад участника проекта, руб;

12 — число месяцев в году, руб;

HPB — годовой фонд рабочего времени, в дн.

В Таблице А.3 приведен расчет заработной платы участников проекта. Количество рабочих дней для каждой роли определяется на основе Таблицы А.1 с учетом участия в выполнении работ.

Таблица А.3 — Расчет основной заработной платы

Наименование этапа	Исполнитель (должность)	Месячный оклад (руб.)	Оплата за день (руб.)	Количество рабочих дней	Оплата за этап (руб.)
1. Формирование требований к проекту	Заказчик системы	120 000	4 800	3	14 400
	Руководитель ВКР	110 000	4 400	3	13 200
	Разработчик проекта	115 000	4 600	4	18 400
2. Разработка концепции проекта	Заказчик системы	120 000	4 800	2	9 600
	Руководитель ВКР	110 000	4 400	2	8 800
	Разработчик проекта	115 000	4 600	8	36 800
3. Техническое задание	Заказчик системы	120 000	4 800	2	9 600
	Разработчик проекта	115 000	4 600	4	18 400
4. Эскизный проект	Разработчик проекта	115 000	4 600	39	179 400
5. Технический проект	Заказчик системы	120 000	4 800	12	57 600
	Разработчик проекта	115 000	4 600	43	197 800

Продолжение Таблицы А.3

6. Рабочая документация	Разработчик проекта	115 000	4 600	15	69 000
	Руководитель ВКР	110 000	4 400	5	22 000
7. Ввод в действие	Заказчик системы	120 000	4 800	7	33 600
	Руководитель ВКР	110 000	4 400	4	17 600
	Разработчик проекта	115 000	4 600	29	133 400
Итого	776 200				

Таким образом, заработная плата исполнителей составит 776 200 рублей.

В среднем расходы по дополнительной заработной плате составляют 20-30% от суммы основной заработной платы. В процессе определения сметы затрат вводится понятие «фонд оплаты труда», представляющее собой сумму основной (ОЗП) и дополнительной заработной платы (ДЗП). Фонд оплаты труда (ФОТ) используется при расчете взносов в социальный фонд. Во всех других случаях (накладные расходы, командировки и др.) расчеты ведутся от базы основной заработной платы.

Расходы на дополнительные заработные платы участников проекта представлены в Таблице А4.

Таблица А.4 — Расчет дополнительной заработной платы

Показатель	Процент, %	Значение (руб.)
Основная заработная плата (ОЗП)	-	776 200
Дополнительная заработная плата (ДЗП)	20	155 240
Фонд оплаты труда (ФОТ)	-	931 440

Таким образом, фонд оплаты труда составляет 931 440 рублей.

В соответствии со статьей 425 НК РФ в 2026 году для лиц, которые производят выплаты и вознаграждения физическим лицам (за исключением плательщиков, для которых установлены пониженные тарифы страховых взносов), установлен Единый тариф страховых взносов 30 % по направлениям:

- обязательное пенсионное страхование (ОПС),

- обязательное социальное страхование на случай временной нетрудоспособности и в связи с материнством (ОСС),
- обязательное медицинское страхование (ОМС).

Таким образом, страховые взносы (СВ) во внебюджетные фонды составят 30,2 % от фонда оплаты труда (ФОТ), который состоит из основной и дополнительной заработной платы, и равны 281 495 рубля.

В процессе проектирования и разработки ИТ-решения также формируются затраты, связанные с приобретением материалов, покупных изделий, комплектующих изделий и других материальных ценностей, расходуемых непосредственно.

Стоимость материалов представлена в Таблице А.5.

Таблица А.5 — Расчет стоимости материалов

Наименование материалов	Единицы измерения	Количество	Цена за единицу (руб.)	Стоимость (руб.)
Флешка (для хранения данных и модели)	шт.	1	700	700
Бумага А 4	пачка	1	500	500
Картридж для принтера	шт.	1	4500	4 500
Канцелярский набор	шт.	1	600	600
Итого материалов				6 300
Транспортно-заготовительные расходы (10 %)				630
Итого:				6 930

Таким образом, сумма затрат по материалам составляет 6 930 руб.

Затраты на комплектующие и изготовление программно-аппаратного комплекса представлены в Таблице А.6.

Таблица А.6 — Расчет стоимости комплектующих

Наименование элементов	Единицы измерения	Количество	Цена за единицу (руб.)	Стоимость (руб.)
Подписка на музыкальные API/сервисы	услуга	1	2 000	2 000
Лицензионное программное обеспечение (средства разработки)	услуга	1	3 000	3 000

Продолжение Таблицы А.6

Облачные вычислительные ресурсы (GPU/CPU для обучения модели)	услуга	1	8 000	8 000
Итого комплектующих				13 000
Транспортно-заготовительные расходы (10 %)				1 300
Итого:				14 300

Таким образом, стоимость комплектующих составляет 14 300 руб. А в целом по статье «Материалы, покупные изделия и полуфабрикаты» с учетом транспортно-заготовительных расходов — 21 230 рублей.

Расходы на приобретение компьютерной техники и программного обеспечения включают в себя стоимость вычислительной техники и программного обеспечения, необходимых как для проектирования и разработки ИТ-решения, так и для последующей его эксплуатации.

В состав технического обеспечения входят средства компьютерной, организационной и коммуникационной техники.

В соответствии с требуемым техническим обеспечением к нему относится: персональный компьютер разработчика, а также вычислительные ресурсы для обработки данных и обучения нейросетевой модели.

Так как ИТ-инфраструктура на объекте исследования уже есть, то затраты по оборудованию будут отсутствовать.

Амортизационные отчисления по используемому в проекте оборудованию определяются на основе балансовой стоимости основных фондов, а также с учетом установленных норм. Начисление амортизации происходит на основные фонды стоимостью от 100 тыс. руб. и выше, если иное не уставлено в учётной политики организации. Способ начисления амортизации также определятся учетной политикой организации. Предполагается, что установлен линейный способ начисления амортизации.

Амортизационные отчисления определяются по формуле:

$$A = \frac{C}{T},$$

(А.2)

где A — сумма амортизационных отчислений за месяц, руб.;

C — первоначальная стоимость объекта, руб.;

T — срок полезного использования, мес.

В Таблице А.7 представлены расчеты амортизационных отчислений по использованному во время разработки проекта оборудованию.

Таблица А.7 — Амортизационные отчисления

Наименование оборудования	Первоначальная стоимость объекта, руб.	Срок полезного использования, мес.	Месячная сумма амортизационных отчислений	Период эксплуатации в месяцах	Сумма, руб.
Персональный компьютер разработчика	120 000	36	3 333	3,5	11 665,5
Ноутбук (при необходимости)	80 000	36	2 222	3,5	7 777
Периферийные устройства (принтер/МФУ)	25 000	60	417	3,5	1 459,5
Итого:					20 902

Таким образом, сумма амортизационных отчислений составят 20 902 руб.

В рамках проектирования и разработки ИТ-решения под эксплуатационными расходами понимаются затраты на электроэнергию.

Из техники в работе могут использоваться компьютеры и принтеры разных моделей с различным энергопотреблением. Первые используются на протяжении всей работы над проектом, и, например, согласно Таблице 4.1 это время составляет 1 120 часов (140 дня по 8 часов в день на одну единицу техники), в то время как время работы принтера будет существенно меньше. Стоимость 1 кВт электроэнергии в Москве равна 8 руб. 00 коп. Расчет эксплуатационных расходов представлен в Таблице А.7.

Таблица А.7 — Расчет эксплуатационных расходов

Наименование техники	Мощность, кВт	Время использования, ч.	Стоимость 1 кВт, руб.	Расходы, руб.
Персональный компьютер	0,4	1 120	8,00	3 584
Компьютер	0,2	560		896
МФУ	0,1	5		4
Примерные общие прочие эксплуатационные расходы:				4 484

Таким образом, расчетные общие прочие эксплуатационные расходы составляют 4 484 рублей.

К накладным расходам (НР) относятся расходы на содержание и ремонт зданий, сооружений, оборудования, инвентаря. Это затраты, сопутствующие основному производству, но не связанные с ним напрямую, не входящие в стоимость труда и материалов.

В рамках реализации проекта это могут быть расходы, связанные с содержанием и арендой площадей, с приобретением расходных материалов для офиса, транспортными расходами, оплатой консультационных, информационных и клиринговых услуг, а также с командировочными расходами.

Сумма данных расходов определяется процентом от суммы основной заработной платы (ОЗП), в данном проекте они принимаются равными 100%.

$$НР = 1 \times 776\,200 = 776\,200 \text{ руб.}$$

Накладные расходы составят 776 200 руб.

Вышеперечисленные статьи затрат и результаты расчетов по ним обобщены в Таблицу А.8.

Полная стоимость проекта составляет 2 050 051 рублей.

Таблица А.8 — Полная себестоимость проекта

Номенклатура статей расходов	Расходы (руб.)	Доля расходов, %
Основная заработная плата	776 200	38,1
Дополнительная заработная плата	155 240	7,6

Продолжение Таблицы А.8

Страховые взносы	281 495	13,8
Материалы, покупные изделия и полуфабрикаты	21 230	1,0
Расходы на приобретение компьютерной техники и программного обеспечения	14 300	0,7
Амортизационные отчисления	20 902	1,0
Эксплуатационные расходы, связанные с использованием компьютерной техники	4 484	0,2
Накладные расходы	776 200	38,1
Прочие	0	0
Итого:	2 050 051	

Для визуализации долевого состава статей затрат в общей себестоимости проекта представлена круговая диаграмма — Рисунок А.3.

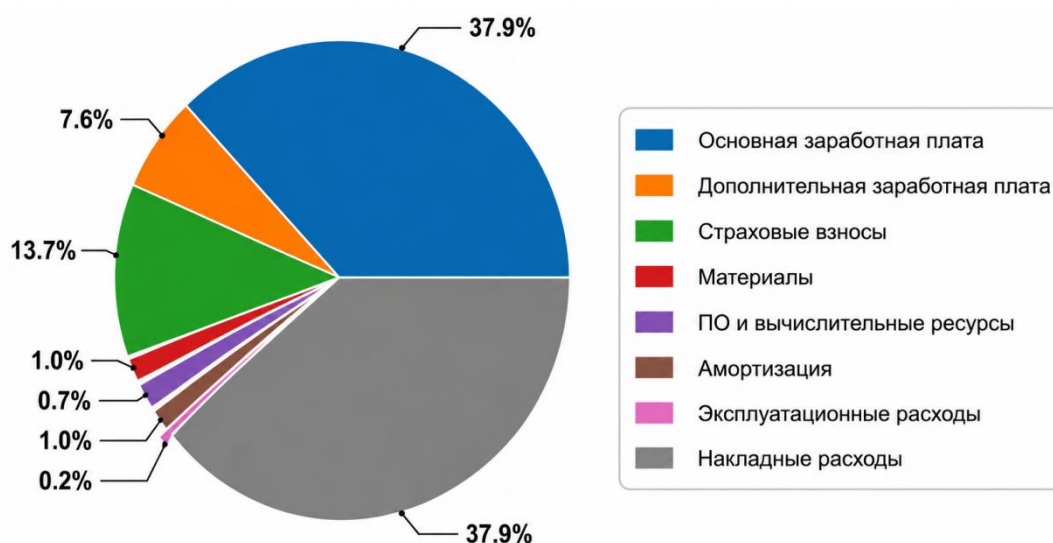


Рисунок А.3 — Структура затрат по проекту

Таким образом, в организационно-экономическом обосновании ВКР было реализовано планирование работ по проектированию и разработке ИТ-решения, а также выполнен расчет себестоимости соответствующего проекта. Полная стоимость проекта составляет 2 050 051 рублей.

Приложение Б

Графический материал (презентация)



-  Музыка существенно влияет на мотивацию и эффективность физической активности
-  Подбор треков вручную не учитывает текущий ритм движения
-  Отсутствуют открытые системы, объединяющие: данные движения + музыкальные характеристики + предпочтения пользователя

Цель и задачи работы



Цель работы: Разработка интеллектуальной системы подбора музыкальных треков по ритму движения пользователя на основе данных акселерометра.

Основные задачи:

1. Анализ данных движения и вычисление BPM активности
2. Анализ музыкальных характеристик треков
3. Разработка модели оценки соответствия движения и музыки
4. Реализация прототипа и оценка качества работы системы

1

Аналоги



Система	Определение BPM движения	Подбор по BPM	Учёт энергичности	Учёт танцевальности	Жанровые предпочтения	Фильтрация по артистам	Доступность технологий
Spotify Running	Да	Да	Нет	Нет	Нет	Нет	Нет
RockMyRun	Нет	Да	Нет	Нет	Нет	Нет	Нет
PaceDJ	Нет	Да	Нет	Нет	Нет	Нет	Да
Разрабатываемая система	Да	Да	Да	Да	Да	Да	Да

1

Данные

- Данные акселерометра (датасет MotionSense)
- Музыкальные характеристики треков
 - Пользовательские предпочтения (жанр, артисты, энергичность)

Модель

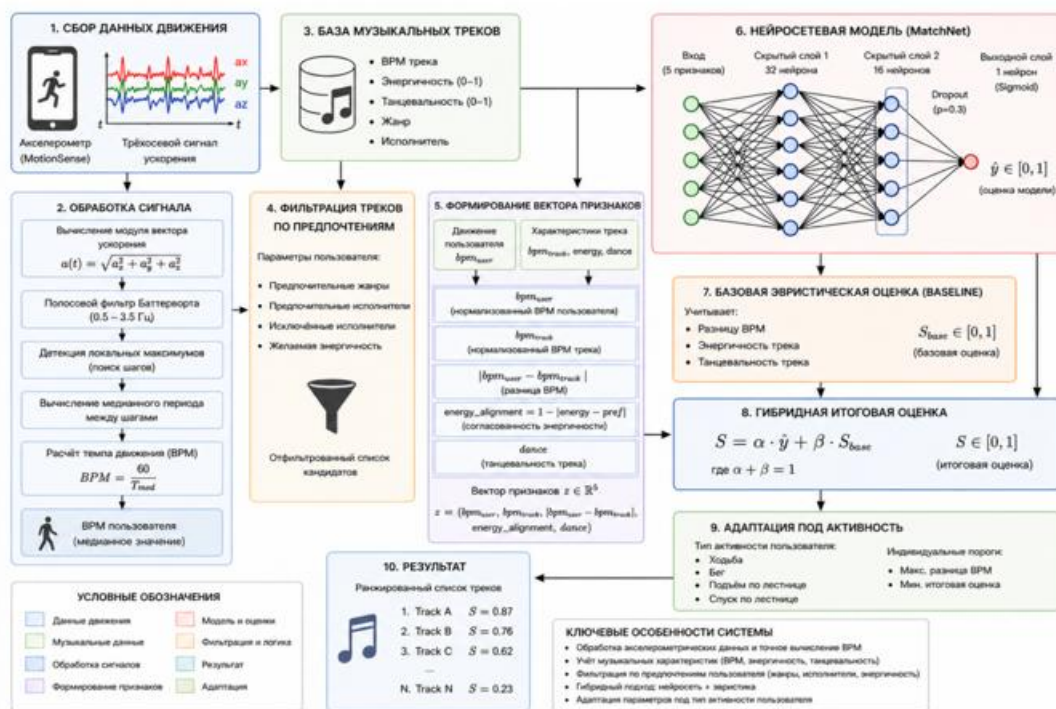
- Нейросетевая модель MatchNet
- Базовая эвристическая оценка
 - Объединение результатов

1

Используемые языки и технологии

-  **NumPy, Pandas** — обработка данных
-  **SciPy** — фильтрация и анализ сигналов
-  **PyTorch** — реализация нейросетевой модели
-  **Streamlit** — пользовательский интерфейс

1



Расчёт надёжности



MSE — среднеквадратическая ошибка

Показывает средний квадрат отклонений предсказанных значений от фактических.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Результат:
0.058



MAE — средняя абсолютная ошибка

Показывает среднее абсолютное отклонение предсказанных значений от фактических.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Результат:
0.192



R^2 — коэффициент детерминации

Показывает долю объяснённой дисперсии. Чем ближе к 1, тем лучше модель объясняет данные.

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Результат:
0.63



r — коэффициент корреляции Пирсона

Показывает силу и направление линейной связи между предсказанными и фактическими значениями.

$$r = \frac{\sum_{i=1}^n (y_i - \bar{y})(\hat{y}_i - \bar{\hat{y}})}{\sqrt{\sum_{i=1}^n (y_i - \bar{y})^2} \sqrt{\sum_{i=1}^n (\hat{y}_i - \bar{\hat{y}})^2}}$$

Результат:
0.79



Модель демонстрирует допустимую ошибку и устойчивую связь между ритмом движения и параметрами музыки.



Демонстрация работы системы

Настройки пользователя

☐ Использовать предыдущий ввод

Тип активности
Ходьба

Жанр музыки
k-pop

Предпочитаемые исполнители
BLACKPINK, TWICE

Исключить исполнителей
BTS

Желаемый стиль треков по интенсивности и танцевальности:

- ☐ Без предпочтений
- ☒ Более энергичные и ритмичные
- ☐ Более спокойные и ненавязчивые

Подобрать треки

BPM рассчитан по нескольким файлам активности: использовано 96, пропущено 1.

Рассчитанный BPM движения: 115.4

Уверенность рекомендаций: Высокая. Лучшие треки хорошо совпадают с темпом движения и проходят строгий отбор.

Рекомендованные треки

	Исполнитель	Трек	Жанр	BPM	Схожесть BPM	Оценка совпадения
0	BLACKPINK	Crazy Over You	k-pop	115	Очень высокая	0.9371
1	TWICE	SCIENTIST	k-pop	113	Очень высокая	0.9301
2	TWICE	Dance The Night Away	k-pop	118.1	Очень высокая	0.9195
3	TWICE	The Feels	k-pop	120	Высокая	0.9031
4	TWICE	Talk that Talk	k-pop	120	Высокая	0.9018
5	BLACKPINK	Forever Young	k-pop	109.8	Высокая	0.8793
6	BLACKPINK	Shut Down	k-pop	110.1	Высокая	0.8639
7	BLACKPINK	Bet You Wanna (Feat. Cardi B)	k-pop	109.8	Высокая	0.8427
8	BLACKPINK	Ready For Love	k-pop	107.6	Средняя	0.8101
9	BLACKPINK	Hope Not	k-pop	110.1	Высокая	0.8158

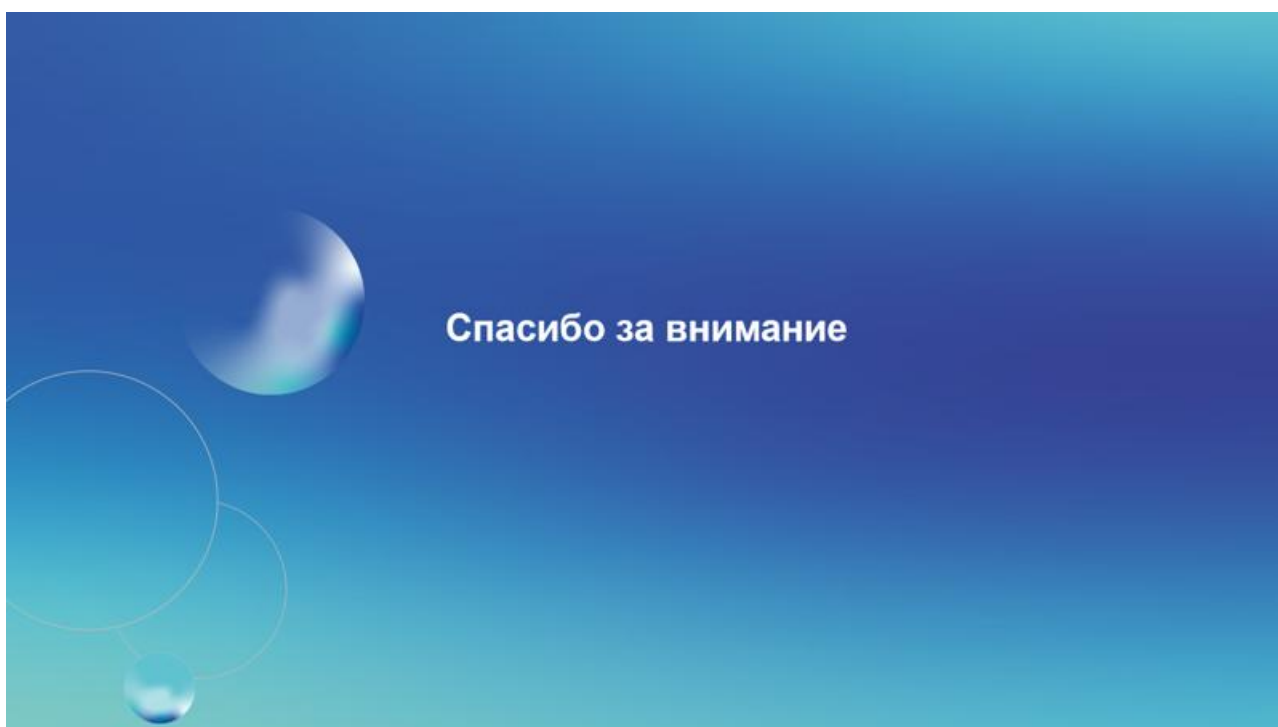
Перспективы развития



- ✓ Реализован анализ данных движения и музыкальных характеристик;
- ✓ Разработана интеллектуальная система подбора музыки по ритму движения;
- ✓ Создан рабочий прототип и подтверждена надёжность системы.

Все задачи посланные перед ВКР выполнены

1



Приложение В

Листинг программного кода

Листинг В.1 – реализация нейросетевой модели MatchNet

```
import torch
import torch.nn as nn

class MatchNet(nn.Module):
    def __init__(self, input_dim=5):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 32),
            nn.ReLU(),
            nn.Dropout(0.2),

            nn.Linear(32, 16),
            nn.ReLU(),
            nn.Dropout(0.2),

            nn.Linear(16, 1),
            nn.Sigmoid()
        )

    def forward(self, x):
        return self.net(x)
```

Листинг В.2 – реализация алгоритма обучения нейросетевой модели MatchNet

```
import os
import sys
from typing import Iterable

import numpy as np
import pandas as pd
import torch
import torch.nn as nn
from scipy.signal import butter, filtfilt, find_peaks
from sklearn.model_selection import train_test_split
from torch.utils.data import DataLoader, TensorDataset,
WeightedRandomSampler

# Добавляем корень проекта в путь импорта, чтобы можно было подключить
# модель.
sys.path.append(os.path.dirname(os.path.dirname(__file__)))

from models.matchnet import MatchNet

def encode_energy_pref(pref: str) -> float:
    pref = pref.lower()
    if pref == "high":
        return 1.0
    if pref == "low":
        return 0.0
    return 0.5

def build_features( # создает признаки для модели
```

Продолжение листинга В.2

```
        обязательные колонки
        missing = [c for c in required if c not in df.columns] # ищет, каких
        колонок не хватает
        if missing:
            raise ValueError(f"Нет обязательных колонок: {missing}")

        df = df.dropna(subset=required).copy() # удаляет строки, где есть
        пропуски в обязательных колонках
        df["bpm"] = pd.to_numeric(df["bpm"], errors="coerce")
        df["energy"] = pd.to_numeric(df["energy"], errors="coerce")
        df["dance"] = pd.to_numeric(df["dance"], errors="coerce")
        df = df.dropna(subset=["bpm", "energy", "dance"]).reset_index(drop=True)
        return df

def butter_bandpass(lowcut: float, highcut: float, fs: float, order: int =
2):
    nyq = 0.5 * fs
    low = lowcut / nyq
    high = highcut / nyq

    if high >= 1:
        high = 0.99

    b, a = butter(order, [low, high], btype="band")
    return b, a

def apply_bandpass_filter(
    signal: np.ndarray,
    fs: float,
    low: float,
    high: float,
) -> np.ndarray:
    b, a = butter_bandpass(low, high, fs)
    return filtfilt(b, a, signal)

def extract_acc_magnitude(df: pd.DataFrame) -> np.ndarray:
    candidates = [
        ("userAcceleration.x", "userAcceleration.y", "userAcceleration.z"),
        ("Acc_x", "Acc_y", "Acc_z"),
        ("acc_x", "acc_y", "acc_z"),
    ]

    for cols in candidates:
        if all(c in df.columns for c in cols):
            ax = df[cols[0]].to_numpy(dtype=float)
            ay = df[cols[1]].to_numpy(dtype=float)
            az = df[cols[2]].to_numpy(dtype=float)
            return np.sqrt(ax**2 + ay**2 + az**2)

    raise ValueError("Не удалось найти колонки акселерометра (x, y, z) в
MotionSense CSV.")

def compute_bpm_from_motion_file(
    path: str,
    sample_rate_hz: float = 50.0,
    bandpass_low: float = 0.5,
    bandpass_high: float = 3.5,
) -> float:
```

Продолжение листинга B.2

```
        raise FileNotFoundError(f"Папка MotionSense не найдена: {motionsense_root}")

    user_bpms: list[float] = []
    skipped_files = 0

    for dirpath, _, filenames in os.walk(motionsense_root):
        for filename in filenames:
            if not filename.lower().endswith(".csv"):
                continue

            csv_path = os.path.join(dirpath, filename)
            try:
                bpm = compute_bpm_from_motion_file(csv_path)
            except Exception:
                skipped_files += 1
                continue

            user_bpms.append(bpm)

    if not user_bpms:
        raise RuntimeError("Не удалось вычислить ни одного BPM из MotionSense CSV.")

    print(
        f"Загружено BPM из MotionSense: {len(user_bpms)}. "
        f"Пропущено файлов: {skipped_files}"
    , flush=True)
    return user_bpms

def normalize_features(X: np.ndarray) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    # нормализуем только BPM-признаки, потому что energy и dance уже
    # примерно в диапазоне 0..1
    means = X.mean(axis=0)
    stds = X.std(axis=0)

    normalize_idx = [0, 1, 2]
    stds[normalize_idx] = np.where(stds[normalize_idx] < 1e-8, 1.0, stds[normalize_idx])
    X[:, normalize_idx] = (X[:, normalize_idx] - means[normalize_idx]) / stds[normalize_idx]
    return X, means, stds

def build_sampler(labels: Iterable[float]) -> WeightedRandomSampler:
    labels_array = np.asarray(list(labels), dtype=np.float32)
    class_counts = np.bincount(labels_array.astype(int), minlength=2)
    class_weights = np.zeros(2, dtype=np.float64)

    for class_idx, count in enumerate(class_counts):
        class_weights[class_idx] = 0.0 if count == 0 else 1.0 / float(count)

    sample_weights = class_weights[labels_array.astype(int)]
    return WeightedRandomSampler(
        weights=torch.as_tensor(sample_weights, dtype=torch.double),
        num_samples=len(sample_weights),
        replacement=True,
    )
```

Продолжение листинга В.2

```
def compress_user_bpms(user_bpms: list[float], max_points: int = 80) ->
list[float]:
    # убираем почти одинаковые BPM, чтобы обучение не разрасталось из-за
    # сотен близких MotionSense-файлов
    unique_bpms = np.unique(np.round(np.asarray(user_bpms,
dtype=np.float32), 1))
    if len(unique_bpms) <= max_points:
        return unique_bpms.astype(float).tolist()

    sampled_idx = np.linspace(0, len(unique_bpms) - 1, num=max_points,
dtype=int)
    return unique_bpms[sampled_idx].astype(float).tolist()

def evaluate_epoch(preds: torch.Tensor, targets: torch.Tensor) ->
tuple[float, float]:
    pred_labels = (preds >= 0.5).float()
    accuracy = (pred_labels == targets).float().mean().item()

    positives = (targets == 1).float().sum().item()
    true_positives = ((pred_labels == 1) & (targets ==
1)).float().sum().item()
    recall = 0.0 if positives == 0 else true_positives / positives
    return accuracy, recall

def evaluate_model(
    model: MatchNet,
    loader: DataLoader,
    pos_weight_value: float,
) -> tuple[float, float, float]:
    model.eval()
    total_loss = 0.0
    all_preds = []
    all_targets = []

    with torch.no_grad():
        for batch_x, batch_y in loader:
            preds = model(batch_x)
            weights = torch.where(
                batch_y == 1,
                torch.full_like(batch_y, pos_weight_value),
                torch.ones_like(batch_y),
            )
            loss = nn.functional.binary_cross_entropy(preds, batch_y,
weight=weights)
            total_loss += loss.item()
            all_preds.append(preds)
            all_targets.append(batch_y)

    preds_cat = torch.cat(all_preds, dim=0)
    targets_cat = torch.cat(all_targets, dim=0)
    accuracy, recall = evaluate_epoch(preds_cat, targets_cat)
    avg_loss = total_loss / max(1, len(loader))
    return avg_loss, accuracy, recall

def main(): # точка входа в скрипт
    root_dir = os.path.dirname(os.path.dirname(__file__)) # определяет
    # корень проекта как папку на уровень выше training
    data_path = os.path.join(root_dir, "data",
    "spotify_tracks_for_app_unique.csv") # указывает на CSV с данными
```

Продолжение листинга В.2

```
model_path = os.path.join(root_dir, "models", "matchnet.pt") #
указывает на файл модели
stats_path = os.path.join(root_dir, "models", "matchnet_stats.pt") #
указывает на файл со статистикой нормализации
motionsense_root = os.path.join(root_dir, "data", "A_DeviceMotion_data",
"A_DeviceMotion_data")

df = load_music(data_path)
user_bpms = compress_user_bpms(load_user_bpms(motionsense_root)) #
берет BPM из MotionSense и сжимает близкие значения
prefs = ["high", "low", "neutral"] # варианты предпочтения по энергии

print(f"После сжатия используем {len(user_bpms)} опорных BPM.",
flush=True)

track_bpms = df["bpm"].to_numpy(dtype=np.float32)
energies = df["energy"].to_numpy(dtype=np.float32)
dances = df["dance"].to_numpy(dtype=np.float32)

X_blocks: list[np.ndarray] = [] # будущие признаки
y_blocks: list[np.ndarray] = [] # будущие метки

for user_bpm in user_bpms:
    user_bpm_arr = np.full_like(track_bpms, fill_value=float(user_bpm),
dtype=np.float32)
    bpm_diff = np.abs(user_bpm_arr - track_bpms)

    for pref in prefs:
        pref_val = np.float32(encode_energy_pref(pref))
        energy_alignment = 1.0 - np.abs(energies - pref_val)
        labels = ((bpm_diff <= 12.0) & (np.abs(energies - pref_val) <=
0.45)).astype(np.float32) # формирует целевую метку по правилам

        X_blocks.append(
            np.column_stack(
                [
                    user_bpm_arr,
                    track_bpms,
                    bpm_diff,
                    energy_alignment.astype(np.float32),
                    dances,
                ]
            ).astype(np.float32)
        )
        y_blocks.append(labels)

X_np = np.vstack(X_blocks) # объединяет все признаки в одну матрицу
y_np = np.concatenate(y_blocks) # объединяет все метки в один вектор

X_np, means, stds = normalize_features(X_np) # нормализует BPM-признаки

positives = int((y_np == 1).sum())
negatives = int((y_np == 0).sum())
positive_ratio = float(positives / len(y_np))

print(
    f"Всего примеров: {len(y_np)} | "
    f"Положительных: {positives} | "
    f"Отрицательных: {negatives} | "
    f"positive_ratio: {positive_ratio:.4f}"
, flush=True)
```

Продолжение листинга В.2

```
        indices = np.arange(len(y_np))
        train_idx, val_idx = train_test_split(
            indices,
            test_size=0.2,
            random_state=42,
            stratify=y_np.astype(int),
        )

        X_train = torch.tensor(X_np[train_idx], dtype=torch.float32) #
# превращает признаки train в тензор
        y_train = torch.tensor(y_np[train_idx].reshape(-1, 1),
                                dtype=torch.float32) # превращает метки train в тензор
        X_val = torch.tensor(X_np[val_idx], dtype=torch.float32) # превращает
# признаки validation в тензор
        y_val = torch.tensor(y_np[val_idx].reshape(-1, 1), dtype=torch.float32)
# превращает метки validation в тензор

        train_positive_ratio = float((y_np[train_idx] == 1).mean())
        val_positive_ratio = float((y_np[val_idx] == 1).mean())
        print(
            f"train_size: {len(train_idx)} | val_size: {len(val_idx)} | "
            f"train_positive_ratio: {train_positive_ratio:.4f} | "
            f"val_positive_ratio: {val_positive_ratio:.4f}",
            flush=True,
        )

        train_dataset = TensorDataset(X_train, y_train)
        val_dataset = TensorDataset(X_val, y_val)
        sampler = build_sampler(y_np[train_idx]) # балансирует классы только на
train
        train_loader = DataLoader(train_dataset, batch_size=128,
sampler=sampler)
        val_loader = DataLoader(val_dataset, batch_size=512, shuffle=False)

        train_positives = int((y_np[train_idx] == 1).sum())
        train_negatives = int((y_np[train_idx] == 0).sum())

        model = MatchNet(input_dim=5)
        pos_weight_value = 1.0 if train_positives == 0 else max(1.0,
train_negatives / max(1, train_positives)) # повышает вес положительного
класса
        optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

        epochs = int(os.environ.get("MATCHNET_EPOCHS", "15")) # число эпох
можно задать через переменную окружения
        for epoch in range(epochs):
            model.train()
            total_loss = 0.0

            for batch_x, batch_y in train_loader:
                optimizer.zero_grad()
                preds = model(batch_x)
                weights = torch.where(
                    batch_y == 1,
                    torch.full_like(batch_y, pos_weight_value),
                    torch.ones_like(batch_y),
                )
                loss = nn.functional.binary_cross_entropy(preds, batch_y,
weight=weights)
                loss.backward()
                optimizer.step()
```

Продолжение листинга В.2

```
        total_loss += loss.item()
    train_loss = total_loss / max(1, len(train_loader))
    val_loss, val_accuracy, val_recall = evaluate_model(model,
val_loader, pos_weight_value)

    print(
        f"Эпоха {epoch + 1}/{epochs} | "
        f"train_loss: {train_loss:.4f} | "
        f"val_loss: {val_loss:.4f} | "
        f"val_accuracy: {val_accuracy:.4f} | "
        f"val_recall: {val_recall:.4f} | "
        f"train_positive_ratio: {train_positive_ratio:.4f} | "
        f"val_positive_ratio: {val_positive_ratio:.4f}"
    , flush=True)

    os.makedirs(os.path.dirname(model_path), exist_ok=True)
    torch.save(model.state_dict(), model_path) # сохраняет веса модели
    torch.save(
        {
            "feature_indices": [0, 1, 2],
            "mean": means[[0, 1, 2]].tolist(),
            "std": stds[[0, 1, 2]].tolist(),
            "means": means.tolist(),
            "stds": stds.tolist(),
            "normalize_idx": [0, 1, 2],
        },
        stats_path, # сохраняет статистику нормализации
    )

    print(f"Модель сохранена: {model_path}", flush=True)
    print(f"Статистика нормализации сохранена: {stats_path}", flush=True)

if __name__ == "__main__":
    main()
```

Листинг В.3 – реализация алгоритма формирования рекомендаций музыкальных треков

```
import json
import os
import sys

from dataclasses import dataclass
from typing import Dict, List, Optional

import numpy as np
import pandas as pd
import streamlit as st
import torch
from scipy.signal import butter, filtfilt, find_peaks

CURRENT_DIR = os.path.dirname(os.path.abspath(__file__))
ROOT_DIR = os.path.dirname(CURRENT_DIR)
MODELS_DIR = os.path.join(ROOT_DIR, "models")

print("CURRENT_DIR =", CURRENT_DIR)
print("ROOT_DIR =", ROOT_DIR)
print("MODELS_DIR =", MODELS_DIR)
print("models exists =", os.path.isdir(MODELS_DIR))

if MODELS_DIR not in sys.path:
```

Продолжение листинга В.3

```
sys.path.insert(0, MODELS_DIR)

from matchnet import MatchNet

@dataclass
class Config:
    motionsense_root: str = os.path.join(ROOT_DIR, "data",
"A_DeviceMotion_data")
    music_csv: str = os.path.join(ROOT_DIR, "data",
"spotify_tracks_for_app_unique.csv")
    model_path: str = os.path.join(ROOT_DIR, "models", "matchnet.pt")
    stats_path: str = os.path.join(ROOT_DIR, "models", "matchnet_stats.pt")
    preferences_path: str = os.path.join(ROOT_DIR, "app",
"user_preferences.json")

    sample_rate_hz: float = 50.0
    bandpass_low: float = 0.5
    bandpass_high: float = 3.5

    bpm_tolerance_max: float = 40.0
    top_n_default: int = 10
    tiny_model_score_threshold: float = 1e-4
    tiny_model_score_share: float = 0.8
    model_weight: float = 0.20
    baseline_weight: float = 0.80

CFG = Config()

def list_activity_files(root: str) -> Dict[str, List[str]]:
    activity_files: Dict[str, List[str]] = {}

    if not os.path.isdir(root):
        raise FileNotFoundError(f"Папка MotionSense не найдена: {root}")

    for dirpath, _, filenames in os.walk(root):
        folder = os.path.basename(dirpath).lower()
        if not filenames:
            continue

        activity_code: Optional[str] = None
        for code in ["wlk", "jog", "run", "ups", "dws"]:
            if code in folder:
                activity_code = code
                break

        if activity_code is None:
            continue

        for f in filenames:
            if f.lower().endswith(".csv"):
                full_path = os.path.join(dirpath, f)
                activity_files.setdefault(activity_code,
[]).append(full_path)

        return activity_files

def butter_bandpass(lowcut: float, highcut: float, fs: float, order: int =
2):
```

Продолжение листинга В.3

```
        nyq = 0.5 * fs
        low = lowcut / nyq
        high = highcut / nyq

        if high >= 1:
            high = 0.99

        b, a = butter(order, [low, high], btype="band")
        return b, a

def apply_bandpass_filter(signal: np.ndarray, fs: float, low: float, high:
float) -> np.ndarray:
    b, a = butter_bandpass(low, high, fs)
    return filtfilt(b, a, signal)

def load_motion_csv(path: str) -> pd.DataFrame:
    if not os.path.isfile(path):
        raise FileNotFoundError(f"CSV MotionSense не найден: {path}")
    return pd.read_csv(path)

def extract_acc_magnitude(df: pd.DataFrame) -> np.ndarray:
    candidates = [
        ("userAcceleration.x", "userAcceleration.y", "userAcceleration.z"),
        ("Acc_x", "Acc_y", "Acc_z"),
        ("acc_x", "acc_y", "acc_z"),
    ]

    for cols in candidates:
        if all(c in df.columns for c in cols):
            ax = df[cols[0]].to_numpy(dtype=float)
            ay = df[cols[1]].to_numpy(dtype=float)
            az = df[cols[2]].to_numpy(dtype=float)
            return np.sqrt(ax ** 2 + ay ** 2 + az ** 2)

    raise ValueError("Не удалось найти колонки акселерометра (x, y, z) в
MotionSense CSV.")

def compute_bpm_from_motion_file(path: str, cfg: Config) -> float:
    df = load_motion_csv(path)
    mag = extract_acc_magnitude(df)
    mag = mag - np.mean(mag)

    filtered = apply_bandpass_filter(
        mag,
        fs=cfg.sample_rate_hz,
        low=cfg.bandpass_low,
        high=cfg.bandpass_high,
    )

    min_distance = int(cfg.sample_rate_hz * 0.3)
    peaks, _ = find_peaks(filtered, distance=min_distance)

    if len(peaks) < 2:
        raise RuntimeError("Недостаточно шагов в сигнале для оценки BPM.")

    intervals = np.diff(peaks)
    median_interval = np.median(intervals)
```

Продолжение листинга В.3

```
        if median_interval <= 0:
            raise RuntimeError("Ошибка при оценке интервала между шагами.")

        step_period_sec = median_interval / cfg.sample_rate_hz
        bpm = 60.0 / step_period_sec
        return float(bpm)

def compute_activity_bpm(file_paths: List[str], cfg: Config) -> tuple[float,
int, int]:
    bpm_values = []
    skipped_files = 0

    for path in file_paths:
        try:
            bpm_values.append(compute_bpm_from_motion_file(path, cfg))
        except Exception:
            skipped_files += 1

    if not bpm_values:
        raise RuntimeError("Не удалось вычислить BPM ни для одного файла
выбранной активности.")

    median_bpm = float(np.median(bpm_values))
    return median_bpm, len(bpm_values), skipped_files

def load_music_database(csv_path: str) -> pd.DataFrame:
    if not os.path.isfile(csv_path):
        raise FileNotFoundError(f"Музыкальный каталог не найден:
{csv_path}")

    df = pd.read_csv(csv_path)
    required_cols = ["title", "artist", "genre", "bpm", "energy", "dance"]
    missing = [c for c in required_cols if c not in df.columns]
    if missing:
        raise ValueError(f"В музыкальной базе отсутствуют обязательные
колонки: {missing}")

    df = df.dropna(subset=required_cols).copy()

    df["bpm"] = pd.to_numeric(df["bpm"], errors="coerce")
    df["energy"] = pd.to_numeric(df["energy"], errors="coerce")
    df["dance"] = pd.to_numeric(df["dance"], errors="coerce")
    df = df.dropna(subset=["bpm", "energy", "dance"]).copy()

    df["artist"] = df["artist"].astype(str).str.strip()
    df["title"] = df["title"].astype(str).str.strip()
    df["genre"] = df["genre"].astype(str).str.strip()

    if "track_id" in df.columns:
        df = df.drop_duplicates(subset=["track_id"], keep="first")
    else:
        df = df.drop_duplicates(subset=["artist", "title"], keep="first")

    return df.reset_index(drop=True)

def filter_by_genre(df: pd.DataFrame, genre_choice: str) -> pd.DataFrame:
    if genre_choice == "Любой жанр":
        return df.copy()
```

Продолжение листинга В.3

```
        mask = df["genre"].str.lower() == genre_choice.lower()
        filtered = df[mask]

        if filtered.empty:
            st.warning("Для выбранного жанра не найдено треков. Используются все треки.")
            return df.copy()

        return filtered.reset_index(drop=True)

def filter_by_artists(
    df: pd.DataFrame,
    include_artists: List[str],
    exclude_artists: List[str],
) -> pd.DataFrame:
    result = df.copy()

    if include_artists:
        result = result[result["artist"].isin(include_artists)]

    if exclude_artists:
        result = result[~result["artist"].isin(exclude_artists)]

    return result.reset_index(drop=True)

def load_user_preferences(path: str) -> dict:
    if not os.path.isfile(path):
        return {}

    try:
        with open(path, "r", encoding="utf-8") as file:
            data = json.load(file)
    except (OSError, json.JSONDecodeError):
        return {}

    return data if isinstance(data, dict) else {}

def save_user_preferences(path: str, preferences: dict) -> None:
    os.makedirs(os.path.dirname(path), exist_ok=True)
    with open(path, "w", encoding="utf-8") as file:
        json.dump(preferences, file, ensure_ascii=False, indent=2)

def sanitize_saved_multiselect(values: List[str], available_options: List[str]) -> List[str]:
    available_set = set(available_options)
    return [value for value in values if value in available_set]

def encode_energy_pref(energy_pref: str) -> float:
    energy_pref = energy_pref.lower()

    if "энергич" in energy_pref:
        return 1.0
    if "спокой" in energy_pref:
        return 0.0
    return 0.5
```

Продолжение листинга В.3

```
def build_match_features(
    user_bpm: float,
    track_bpm: float,
    energy: float,
    dance: float,
    energy_pref: str,
) -> List[float]:
    pref_val = encode_energy_pref(energy_pref)
    energy_alignment = 1.0 - abs(float(energy) - pref_val)

    return [
        float(user_bpm),
        float(track_bpm),
        float(abs(user_bpm - track_bpm)),
        float(energy_alignment),
        float(dance),
    ]

def load_normalization_stats(stats_path: str) -> Optional[dict]:
    if not os.path.isfile(stats_path):
        return None

    stats = torch.load(stats_path, map_location="cpu")
    if {"feature_indices", "mean", "std"}.issubset(stats):
        return stats

    # Совместимость с новым форматом из train_matchnet.py.
    if {"normalize_idx", "means", "stds"}.issubset(stats):
        normalize_idx = list(stats["normalize_idx"])
        means = list(stats["means"])
        stds = list(stats["stds"])
        return {
            "feature_indices": normalize_idx,
            "mean": [means[idx] for idx in normalize_idx],
            "std": [stds[idx] for idx in normalize_idx],
        }

    return None

def apply_feature_normalization(features: List[float], stats:
Optional[dict]) -> List[float]:
    if not stats:
        return features

    normalized = list(features)
    feature_indices = stats["feature_indices"]
    mean = stats["mean"]
    std = stats["std"]

    for offset, feature_index in enumerate(feature_indices):
        denom = std[offset] if std[offset] != 0 else 1.0
        normalized[feature_index] = (normalized[feature_index] -
mean[offset]) / denom

    return normalized

def load_matchnet_model(model_path: str) -> MatchNet:
    if not os.path.isfile(model_path):
        raise FileNotFoundError(
```

Продолжение листинга В.3

```
        "Файл модели MatchNet не найден.\n"
        f"Ожидаемый путь: {model_path}\n\n"
        "Сначала обучите модель командой:\n"
        "python training/train_matchnet.py"
    )

    model = MatchNet(input_dim=5)
    state_dict = torch.load(model_path, map_location="cpu")
    model.load_state_dict(state_dict)
    model.eval()
    return model

def predict_match_score(
    model: MatchNet,
    user_bpm: float,
    track_bpm: float,
    energy: float,
    dance: float,
    energy_pref: str,
    norm_stats: Optional[dict],
) -> float:
    features = build_match_features(
        user_bpm=user_bpm,
        track_bpm=track_bpm,
        energy=energy,
        dance=dance,
        energy_pref=energy_pref,
    )
    features = apply_feature_normalization(features, norm_stats)

    x = torch.tensor(features, dtype=torch.float32).unsqueeze(0)

    with torch.no_grad():
        score = model(x).item()

    return float(score)

def blend_match_scores(model_score: float, baseline_score: float) -> float:
    return CFG.model_weight * float(model_score) + CFG.baseline_weight *
float(baseline_score)

def get_activity_rules(activity_code: str) -> dict:
    rules = {
        "wlk": {"max_bpm_diff": 8.0, "min_match_score": 0.45},
        "jog": {"max_bpm_diff": 8.0, "min_match_score": 0.40},
        "ups": {"max_bpm_diff": 9.0, "min_match_score": 0.42},
        "dws": {"max_bpm_diff": 9.0, "min_match_score": 0.42},
        "run": {"max_bpm_diff": 10.0, "min_match_score": 0.38},
    }
    return rules.get(activity_code, {"max_bpm_diff": 12.0,
"min_match_score": 0.40})

def is_model_degenerate(model_scores: pd.Series, cfg: Config) -> bool:
    if model_scores.empty:
        return False

    tiny_share = float((model_scores.abs() <
cfg.tiny_model_score_threshold).mean())
```

Продолжение листинга В.3

```
        return tiny_share >= cfg.tiny_model_score_share

def describe_bpm_similarity(bpm_diff: float) -> str:
    bpm_diff = float(bpm_diff)
    if bpm_diff <= 3:
        return "Очень высокая"
    if bpm_diff <= 6:
        return "Высокая"
    if bpm_diff <= 10:
        return "Средняя"
    if bpm_diff <= 15:
        return "Низкая"
    return "Очень низкая"

def format_recommendations_for_display(df: pd.DataFrame) -> pd.DataFrame:
    display_df = df.copy()
    display_df["bpm"] = display_df["bpm"].map(lambda value:
round(float(value), 1))
    display_df["match_score"] = display_df["match_score"].map(lambda value:
round(float(value), 4))
    display_df["Схожесть BPM"] =
display_df["bpm_diff"].map(describe_bpm_similarity)
    display_df = display_df.rename(
        columns={
            "artist": "Исполнитель",
            "title": "Трек",
            "genre": "Жанр",
            "bpm": "BPM",
            "match_score": "Оценка совпадения",
        }
    )
    return display_df

def estimate_recommendation_confidence(recs: pd.DataFrame, activity_code:
str) -> tuple[str, str]:
    if recs.empty:
        return "Низкая", "Подходящие треки не найдены после фильтрации
кандидатов."

    rules = get_activity_rules(activity_code)
    top_score = float(recs.iloc[0]["match_score"])
    mean_diff = float(recs["bpm_diff"].head(min(5, len(recs))).mean())
    min_required = float(rules["min_match_score"])

    if top_score >= max(0.75, min_required + 0.2) and mean_diff <= 6:
        return "Высокая", "Лучшие треки хорошо совпадают с темпом движения и
проходят строгий отбор."
    if top_score >= max(0.55, min_required + 0.1) and mean_diff <= 10:
        return "Средняя", "Есть несколько подходящих кандидатов, но
совпадение по темпу не идеальное."
    return "Низкая", "Даже лучшие треки совпадают с движением умеренно,
поэтому рекомендации менее надёжны."

def compute_match_score_baseline(
    user_bpm: float,
    track_bpm: float,
    energy: float,
    dance: float,
```

Продолжение листинга В.3

```
        energy_pref: str,
        cfg: Config,
    ) -> float:
        bpm_diff = abs(track_bpm - user_bpm)
        bpm_score = 1.0 - min(bpm_diff / cfg.bpm_tolerance_max, 1.0)

        energy_pref = energy_pref.lower()
        if "энергич" in energy_pref:
            energy_score = energy
        elif "спокой" in energy_pref:
            energy_score = 1.0 - energy
        else:
            energy_score = 0.5

        dance_score = dance
        score = 0.75 * bpm_score + 0.15 * energy_score + 0.10 * dance_score
        return float(score)

def recommend_tracks(
    music_df: pd.DataFrame,
    user_bpm: float,
    activity_code: str,
    energy_pref: str,
    top_n: int,
    model: MatchNet,
    norm_stats: Optional[dict],
) -> pd.DataFrame:
    music_df = music_df.copy()
    rules = get_activity_rules(activity_code)

    if "track_id" in music_df.columns:
        music_df = music_df.drop_duplicates(subset=["track_id"],
        keep="first")
    else:
        music_df = music_df.drop_duplicates(subset=["artist", "title"],
        keep="first")

    music_df["bpm_diff"] = (music_df["bpm"].astype(float) -
float(user_bpm)).abs()
    music_df = music_df[music_df["bpm_diff"] <=
float(rules["max_bpm_diff"])]

    if music_df.empty:
        return music_df

    scores = []
    model_scores = []
    baseline_scores = []
    for _, row in music_df.iterrows():
        model_score = predict_match_score(
            model=model,
            user_bpm=user_bpm,
            track_bpm=row["bpm"],
            energy=row["energy"],
            dance=row["dance"],
            energy_pref=energy_pref,
            norm_stats=norm_stats,
        )
        baseline_score = compute_match_score_baseline(
            user_bpm=user_bpm,
            track_bpm=row["bpm"],
```

```

        energy=row["energy"],
        dance=row["dance"],
        energy_pref=energy_pref,
        cfg=CFG,
    )
    score = blend_match_scores(model_score, baseline_score)

    model_scores.append(model_score)
    baseline_scores.append(baseline_score)
    scores.append(score)

    music_df["model_score"] = model_scores
    music_df["baseline_score"] = baseline_scores
    music_df["match_score"] = scores
    music_df = music_df[music_df["match_score"] >=
float(rules["min_match_score"])] .copy()

    if music_df.empty:
        return music_df

    music_df = music_df.sort_values(by="match_score", ascending=False)

    if "track_id" in music_df.columns:
        music_df = music_df.drop_duplicates(subset=["track_id"],
keep="first")
    else:
        music_df = music_df.drop_duplicates(subset=["artist", "title"],
keep="first")

    return music_df.head(top_n).reset_index(drop=True)

def main():
    st.set_page_config(
        page_title="Система подбора треков по ритму движения",
        layout="wide",
    )

    st.sidebar.title("Настройки пользователя")

    try:
        music_df = load_music_database(CFG.music_csv)
    except Exception as e:
        st.error(f"Ошибка при загрузке музыкальной базы: {e}")
        return

    try:
        matchnet_model = load_matchnet_model(CFG.model_path)
    except Exception as e:
        st.error(f"Ошибка при загрузке MatchNet: {e}")
        st.stop()

    norm_stats = load_normalization_stats(CFG.stats_path)

    try:
        activity_files = list_activity_files(CFG.motionsense_root)
    except Exception as e:
        st.error(f"Ошибка при поиске файлов MotionSense: {e}")
        return

    preferences = load_user_preferences(CFG.preferences_path)
    use_saved_preferences = st.sidebar.checkbox(

```

Продолжение листинга В.3

```
        "Использовать предыдущий ввод",
        value=bool(preferences),
    )

    if not activity_files:
        st.error("Не найдены CSV MotionSense ни для одной активности.")
        return

    activity_pairs = [
        ("wlk", "Ходьба"),
        ("jog", "Бег"),
        ("ups", "Подъём по лестнице"),
        ("dws", "Спуск по лестнице"),
    ]

    available_pairs = [(code, label) for code, label in activity_pairs if
code in activity_files]

    if not available_pairs:
        st.error("Не найдено ни одной активности с CSV-файлами
MotionSense.")
        return

    display_labels = [label for _, label in available_pairs]
    code_by_label = {label: code for code, label in available_pairs}
    saved_activity_code = preferences.get("activity_code") if
use_saved_preferences else None
    saved_activity_label = next(
        (label for code, label in available_pairs if code ==
saved_activity_code),
        display_labels[0],
    )
    activity_index = display_labels.index(saved_activity_label)
    selected_label = st.sidebar.selectbox("Тип активности", display_labels,
index=activity_index)
    activity_code = code_by_label[selected_label]

    genre_list = ["Любой жанр"] +
sorted(music_df["genre"].dropna().unique())
    saved_genre = preferences.get("genre_choice", "Любой жанр") if
use_saved_preferences else "Любой жанр"
    if saved_genre not in genre_list:
        saved_genre = "Любой жанр"
    genre_index = genre_list.index(saved_genre)
    genre_choice = st.sidebar.selectbox("Жанр музыки", genre_list,
index=genre_index)

    music_for_artists = filter_by_genre(music_df, genre_choice)
    artist_options = sorted(music_for_artists["artist"].dropna().unique())
    saved_include_artists = sanitize_saved_multiselect(
        preferences.get("include_artists", []) if use_saved_preferences else
[],
        artist_options,
    )
    saved_exclude_artists = sanitize_saved_multiselect(
        preferences.get("exclude_artists", []) if use_saved_preferences else
[],
        artist_options,
    )

    include_artists = st.sidebar.multiselect(
```

```
        "Предпочитаемые исполнители",
        options=artist_options,
        default=saved_include_artists,
    )

    exclude_artists = st.sidebar.multiselect(
        "Исключить исполнителей",
        options=artist_options,
        default=saved_exclude_artists,
    )

    energy_options = [
        "Без предпочтений",
        "Более энергичные и ритмичные",
        "Более спокойные и ненавязчивые",
    ]
    saved_energy_pref = preferences.get("energy_pref", energy_options[0]) if
use_saved_preferences else energy_options[0]
    if saved_energy_pref not in energy_options:
        saved_energy_pref = energy_options[0]
    energy_index = energy_options.index(saved_energy_pref)
    energy_pref = st.sidebar.radio(
        "Желаемый стиль треков по интенсивности и танцевальности:",
        energy_options,
        index=energy_index,
    )

    save_user_preferences(
        CFG.preferences_path,
        {
            "activity_code": activity_code,
            "genre_choice": genre_choice,
            "include_artists": include_artists,
            "exclude_artists": exclude_artists,
            "energy_pref": energy_pref,
        },
    )

    st.title("Интеллектуальная система подбора музыкальных треков по ритму
движения пользователя")
    st.write(
        "Выберите параметры слева и нажмите кнопку ниже, чтобы система
подобрала треки "
        "под ритм вашего движения."
    )

    if st.button("Подобрать треки"):
        try:
            user_bpm, used_files, skipped_files = compute_activity_bpm(
                activity_files[activity_code],
                CFG,
            )
        except Exception as e:
            st.error(f"Ошибка при вычислении BPM: {e}")
            return
        st.info(
            f"BPM рассчитан по нескольким файлам активности: использовано
{used_files}, "
            f"пропущено {skipped_files}."
        )
        st.success(f"Рассчитанный BPM движения: {user_bpm:.1f}")
```

```

        music_filtered = filter_by_genre(music_df, genre_choice)
        music_filtered = filter_by_artists(music_filtered, include_artists,
exclude_artists)
        if music_filtered.empty:
            st.warning("После фильтрации по жанру и исполнителям не осталось
ни одного трека.")
            return
        try:
            recs = recommend_tracks(
                music_df=music_filtered,
                user_bpm=user_bpm,
                activity_code=activity_code,
                energy_pref=energy_pref,
                top_n=CFG.top_n_default,
                model=matchnet_model,
                norm_stats=norm_stats,
            )
        except Exception as e:
            st.error(f"Ошибка при формировании рекомендаций: {e}")
            return
        if recs.empty:
            st.warning(
                "Подходящих треков не найдено после отбора кандидатов. "
                "Попробуйте другой жанр, измените список исполнителей или
выберите другую активность."
            )
            return

        confidence_label, confidence_message =
estimate_recommendation_confidence(recs, activity_code)
        if confidence_label == "Высокая":
            st.success(f"Уверенность рекомендаций: {confidence_label}.
{confidence_message}")
        elif confidence_label == "Средняя":
            st.info(f"Уверенность рекомендаций: {confidence_label}.
{confidence_message}")
        else:
            st.warning(f"Уверенность рекомендаций: {confidence_label}.
{confidence_message}")

        recs_display = format_recommendations_for_display(recs)

        st.subheader("Рекомендованные треки")
        st.dataframe(
            recs_display[
                [
                    "Исполнитель",
                    "Трек",
                    "Жанр",
                    "BPM",
                    "Схожесть BPM",
                    "Оценка совпадения",
                ]
            ],
            use_container_width=True,
        )

    if __name__ == "__main__":
        main()

```

ОТЗЫВ
РУКОВОДИТЕЛЯ ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЫ

Тема ВКР: Интеллектуальная система подбора треков по ритму движения пользователя

Автор (студент /ка) Свирская Варвара Александровна

Институт информационных технологий

Кафедра вычислительной техники

Направление 09.03.04 «Программная инженерия»

Профиль «Интеллектуальные системы поддержки принятия решений»

Руководитель Лозовский Владимир Валерьевич, доцент кафедры вычислительной техники, РТУ

МИРЭА, к.т.н.

(Ф.И.О., место работы, должность, ученое звание, степень)

Оценка соответствия требованиям ФГОС подготовленности автора выпускной работы:

Требования к профессиональной подготовке	Соответствует	В основном соответствует	Не соответствует
уметь корректно формулировать и ставить задачи (проблемы) своей деятельности при выполнении выпускной квалификационной работы, анализировать, диагностировать причины появления проблем, их актуальность	+		
устанавливать приоритеты и методы решения поставленных задач (проблем)	+		
владеть компьютерными методами сбора, хранения и обработки (редактирования) информации	+		
уметь рационально планировать время выполнения работы, определять последовательность и объем операций и решений при выполнении поставленной задачи	+		
уметь объективно оценивать полученные результаты расчетов, вычислений, используя для сравнения данные других направлений соответствующей отрасли науки	+		
уметь анализировать полученные результаты и их интерпретации	+		
уметь осуществлять деятельность в кооперации с коллегами, находить компромиссы	+		
уметь делать самостоятельные обоснованные и достоверные выводы из проделанной работы	+		
уметь пользоваться научной литературой профессиональной направленности	+		

Отмеченные достоинства работа отличается глубиной исследования методов подбора музыкальных рекомендаций на основе анализа двигательной активности пользователя. В работе проведён анализ современных подходов к обработке данных движения и рекомендационных систем, выполнено проектирование архитектуры программного решения, разработана и реализована интеллектуальная система подбора музыкальных треков. Достоинством работы является использование гибридного подхода, сочетающего нейросетевую модель и эвристическую оценку соответствия, а также реализация механизма оценки уровня уверенности рекомендаций.

Отмеченные недостатки в работе отсутствует механизм использования пользовательской обратной связи для дополнительного обучения модели рекомендаций. Также система ограничена используемым предварительно подготовленным набором музыкальных данных. В перспективе возможно внедрение пользовательских оценок и расширение базы треков.

Заключение считаю, что выпускная квалификационная работа Свирскую Варвару Александровну заслуживает высокой оценки, а ее автор достоин присвоения квалификации по направлению 09.03.04 «Программная инженерия» профиль «Интеллектуальные системы поддержки принятия решений».

Руководитель проекта


подпись

Лозовский В.В.
фамилия и инициалы

«28» мая 2026г.

Справка о результатах проверки документа на наличие заимствований



Название документа: Интеллектуальная система подбора треков по ритму движения пользователя

Дата и время проверки: 28.05.2026 19:05:47

Автор: Свирская Варвара Александровна

ВУЗ: МИРЭА – Российский технологический университет

Подразделение: ИИТ

Совпадение 10.85%



Цитирование и самоцитирование 0.12%



Оригинальность 83.93%



Библиография 5.1%



Исключения 0%



Модули поиска: Интернет 2.0, IEEE, Коллекция ЭБС, Диссертации РГБ, Авторефераты РГБ, Wikipedia, НЭБ (сборник научных статей), Сборник юридических документов, Диссертации ВАК, Авторефераты ВАК, Научная электронная библиотека "КиберЛенинка", Научные статьи 2.0, Архив Федерального института промышленной собственности, Объединенная университетская коллекция, Региональный индекс Узбекистана, Национальная библиотека Узбекистана, Коллекция энциклопедий

Проверяющий: Железняк Лилия

Дата: 28.05.2026

Подпись:





МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий
Кафедра вычислительной техники

ЗАКЛЮЧЕНИЕ

ВКР (бакалаврская работа) обучающегося Свирскую Варвару Александровну группы ИКБО-22-22 на тему «Интеллектуальная система подбора треков по ритму движения пользователя» в соответствии с Порядком проведения проверки на объем заимствования и размещения в электронной информационно-образовательной среде РТУ МИРЭА выпускных квалификационных работ (СМКО МИРЭА 8.5.1/03.П.121-25) прошла автоматизированный анализ в системе РУКОНТЕКСТ.

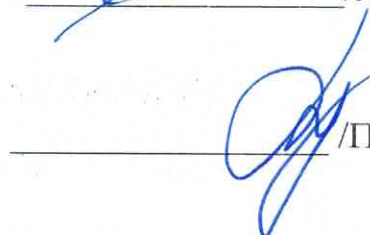
Доля авторского текста (оригинальности) в результате автоматизированной проверки составила 83,93%.

Руководитель ВКР

 /Лозовский В.В./

«28» мая 2026 г.

Заведующий кафедрой

 /Платонова О.В./

«28» мая 2026 г.

МИРЭА – РОССИЙСКИЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ

Председателю Государственной экзаменационной комиссии ИИТ

«01» июня 2026 г.

СПРАВКА

Студент: Свирская Варвара Александровна № зачетной книжки 22И1781

выполнил ВКР на тему: Интеллектуальная система подбора треков по ритму
движения пользователя

и допущен к защите ВКР в Государственной экзаменационной комиссии.

Зав. кафедрой
вычислительной техники

Руководитель ВКР


Платонова О.В.


Лозовский В.В.